Introduction to MySQL

- ♣ Installing MySQL and starting MySQL instance.
- ♣ History of MySQL
- ♣ Components of MySQL -DML,DDL, DCL, DQL
- ♣ Data types in MySQL
- ♣ Creating databases and show databases

Introduction:

MySQL is an open-source relational database management system (RDBMS). It is the most popular database system used with PHP. MySQL is developed, distributed, and supported by Oracle Corporation.

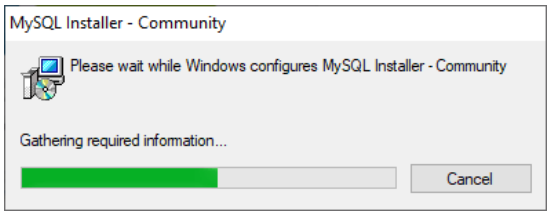
- The data in a MySQL database are stored in tables which consists of columns and rows.
- MySQL is a database system that runs on a server.
- MySQL is ideal for both small and large applications.
- MySQL is very fast, reliable, and easy to use database system. It uses standard SQL
- MySQL compiles on a number of platform

Installing MySQL on Windows

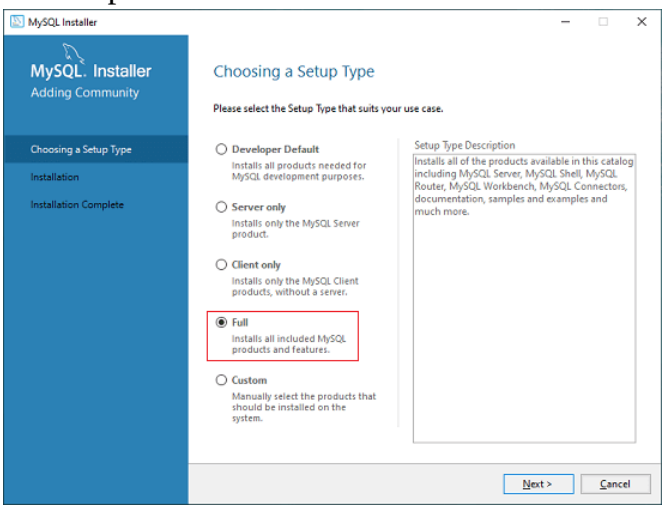
MySQL is one of the most popular relational database management software that is widely used in today's industry. It provides multi-user access support with various storage engines.

- MySQL Setup Software
- Microsoft .NET Framework 4.5.2
- Microsoft Visual C++ Redistributable for Visual Studio 2019
- RAM 4 GB

Step 1: After downloading the setup, double click the MSI **installer .exe file.** It will give the following screen:

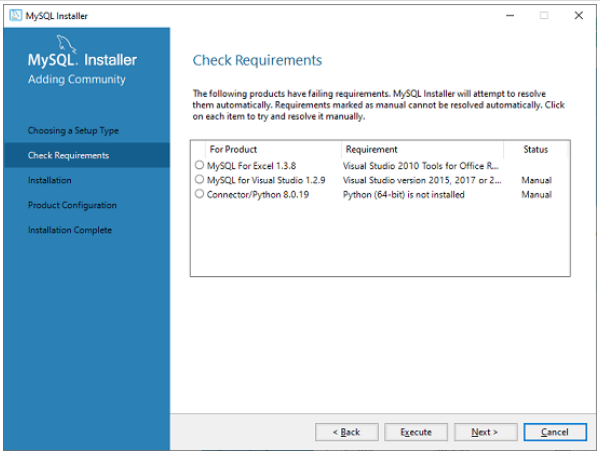


Step 2: In the next wizard, choose the **Setup Type**. There are several types available, and you need to choose the appropriate option to install MySQL product and features. Here, we are going to select the **Full** option and click on the Next button.

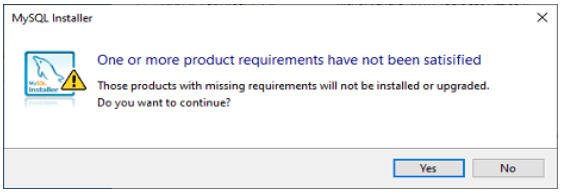


This option will install the following things: MySQL Server, MySQL Shell, MySQL Router, MySQL Workbench, MySQL Connectors, documentation, samples and examples, and many more.

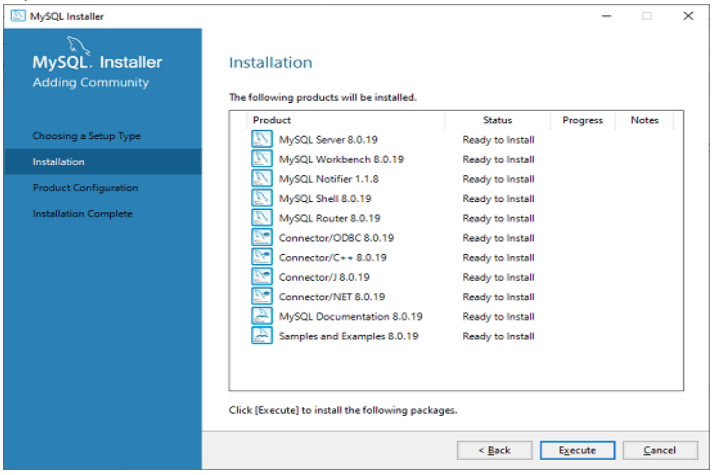
Step 3: Once we click on the Next button, it may give information about some features that may fail to install on your system due to a lack of requirements. We can resolve them by clicking on the **Execute** button that will install all requirements automatically or can skip them. Now, click on the Next button.



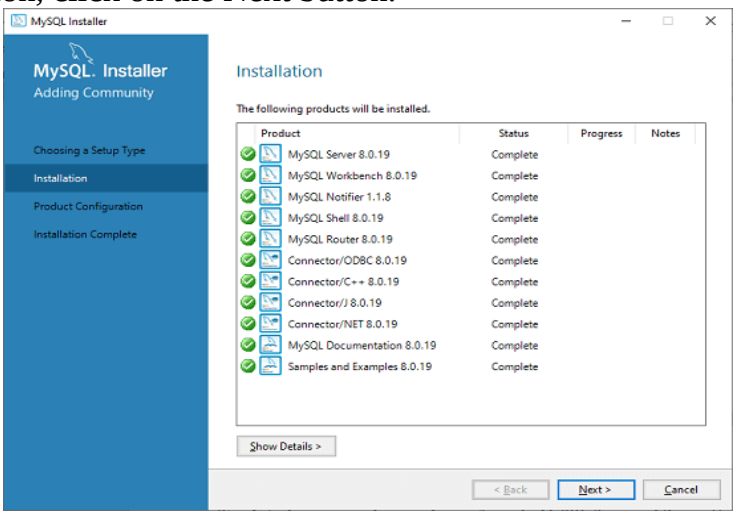
Step 4: In the next wizard, we will see a dialog box that asks for our confirmation of a few products not getting installed. Here, we have to click on the **Yes** button.



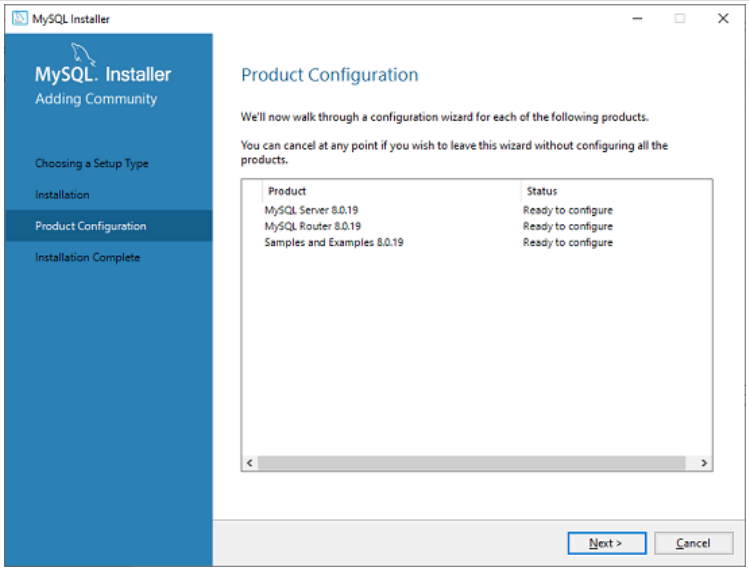
After clicking on the Yes button, we will see the list of the products which are going to be installed. So, if we need all products, click on the Execute button.



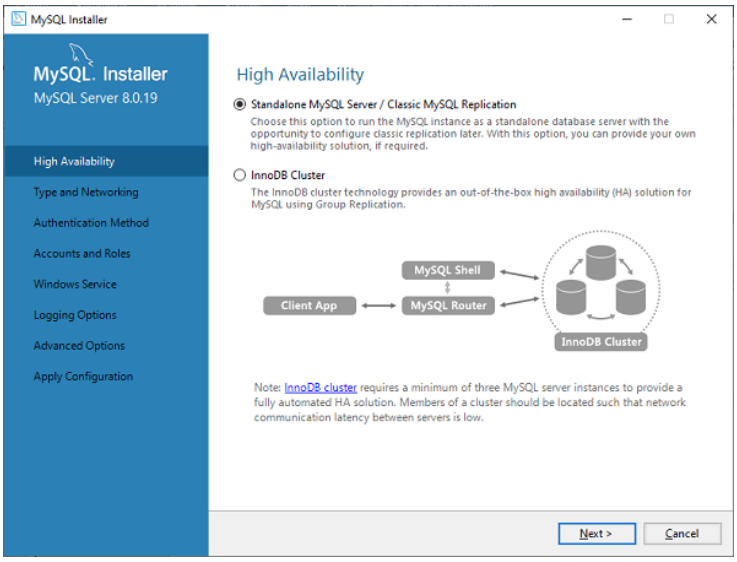
Step 5: Once we click on the Execute button, it will download and install all the products. After completing the installation, click on the Next button.



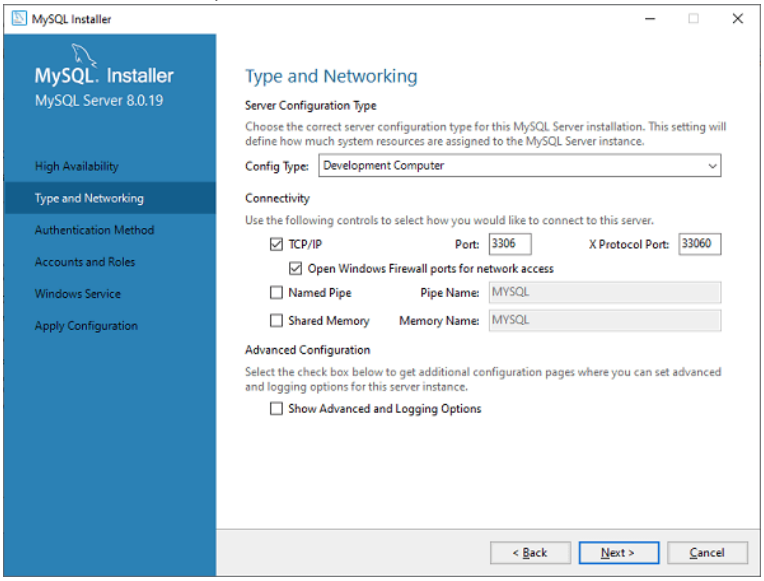
Step 6: In the next wizard, we need to configure the MySQL Server and Router. Here, I am not going to configure the Router because there is no need to use it with MySQL. We are going to show you how to configure the server only. Now, click on the Next button.



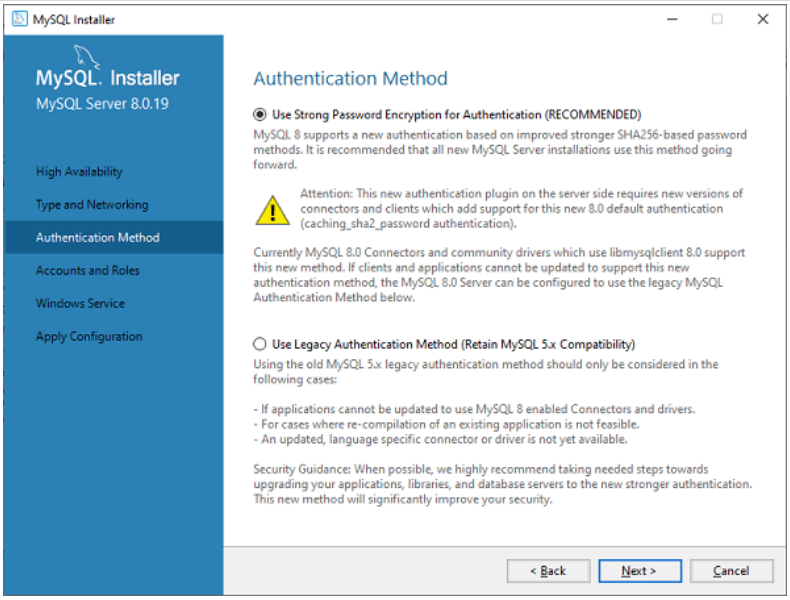
Step 7: As soon as you will click on the Next button, you can see the screen below. Here, we have to configure the MySQL Server. Now, choose the Standalone MySQL Server/Classic MySQL Replication option and click on Next. Here, you can also choose the InnoDB Cluster based on your needs.



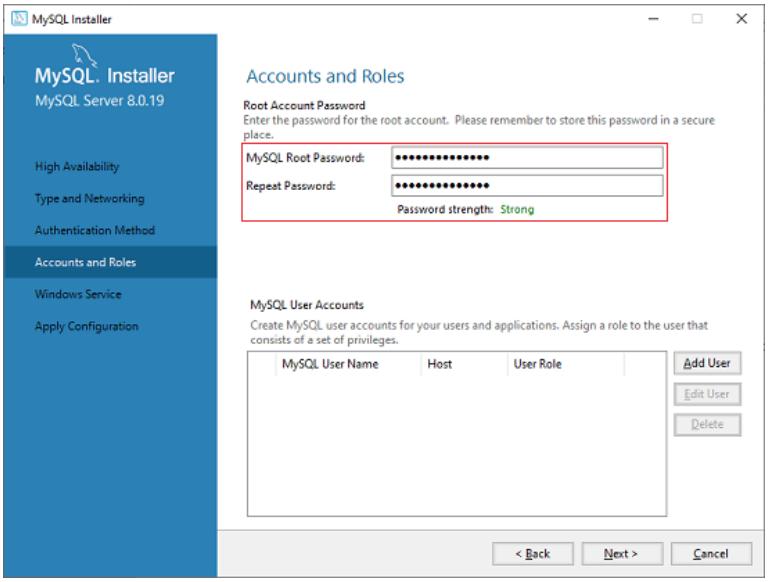
Step 8: In the next screen, the system will ask you to choose the Config Type and other connectivity options. Here, we are going to select the **Config Type** as 'Development Machine' and Connectivity as **TCP/IP,** and **Port Number** is 3306, then click on Next.



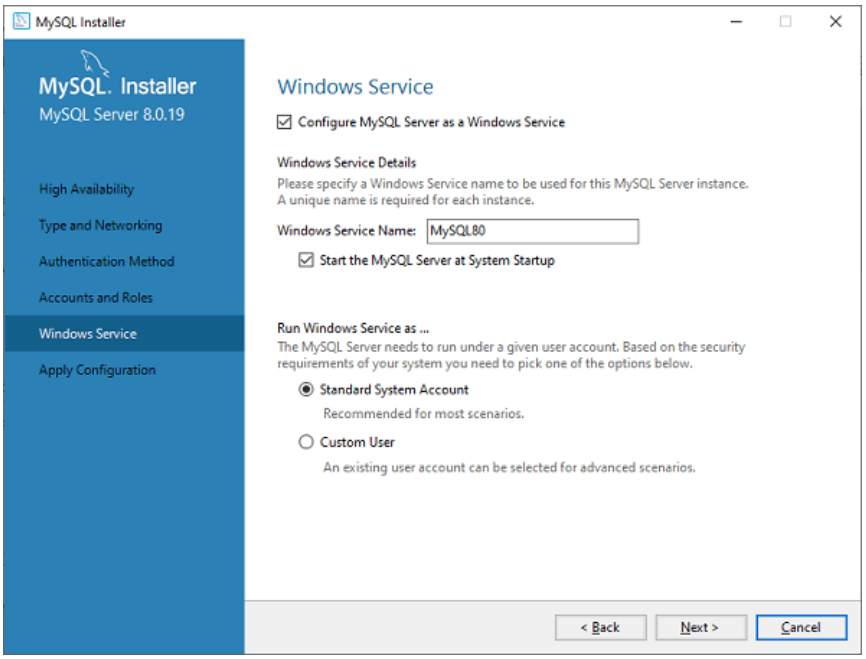
Step 9: Now, select the Authentication Method and click on Next. Here, I am going to select the first option.



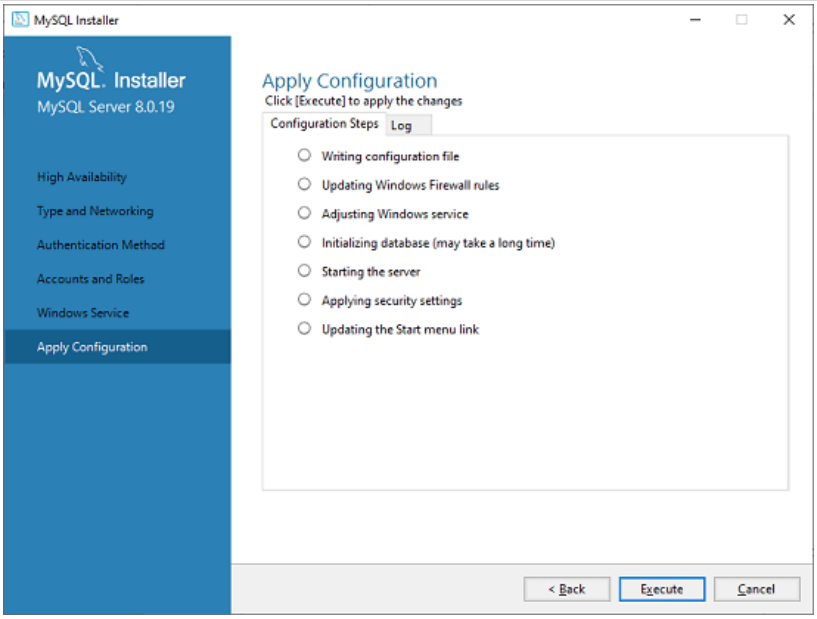
Step 10: The next screen will ask you to mention the MySQL Root Password. After filling the password details, click on the Next button.



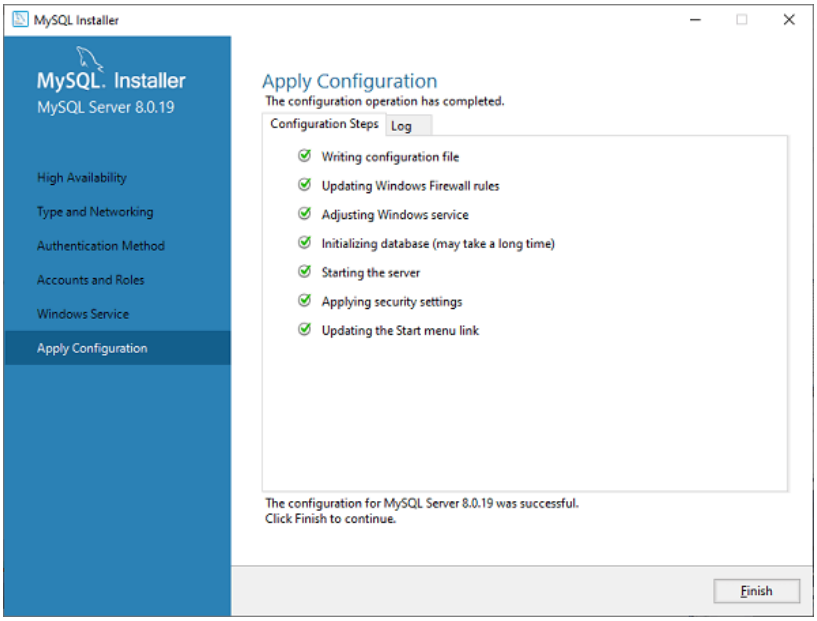
Step 11: The next screen will ask you to configure the Windows Service to start the server. Keep the default setup and click on the Next button.



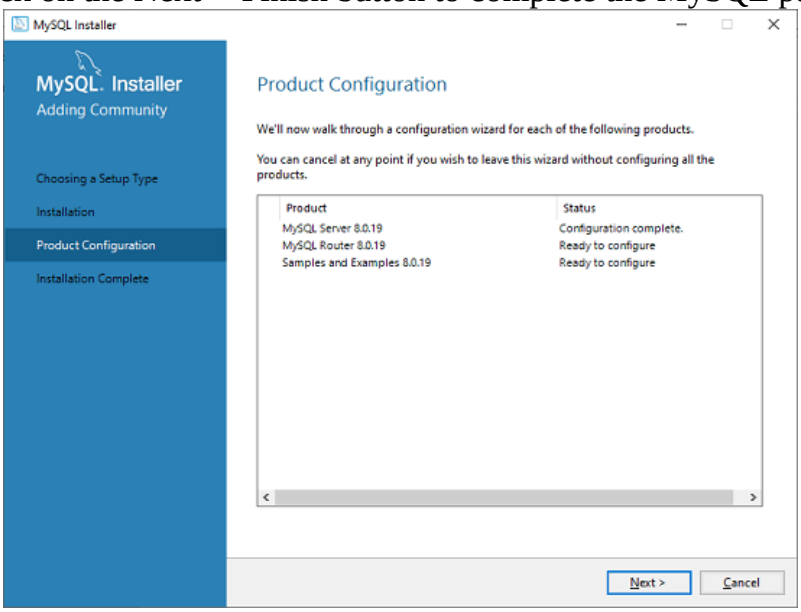
Step 12: In the next wizard, the system will ask you to apply the Server Configuration. If you agree with this configuration, click on the Execute button.



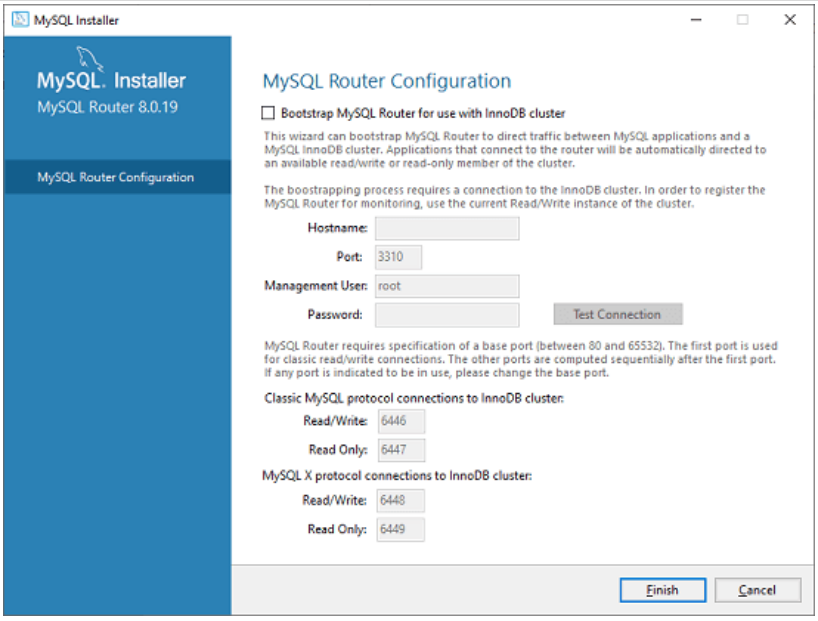
Step 13: Once the configuration has completed, you will get the screen below. Now, click on the **Finish** button to continue.



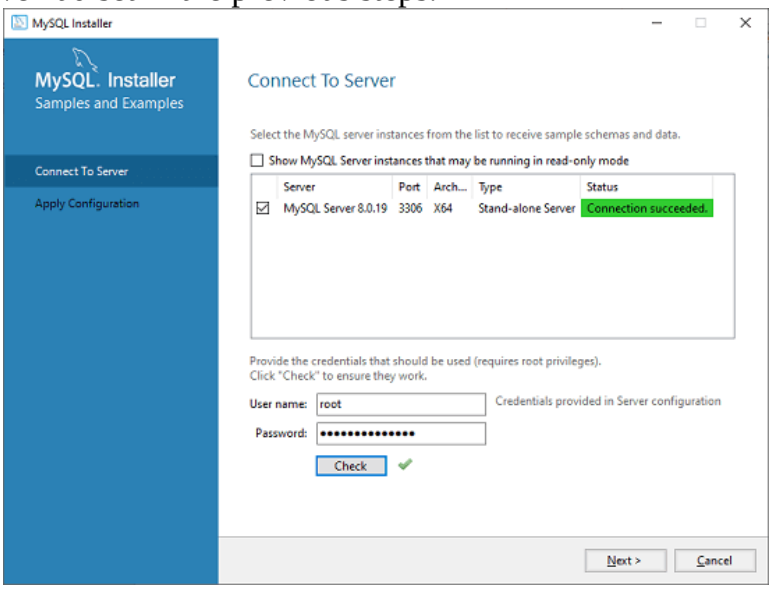
Step 14: In the next screen, you can see that the Product Configuration is completed. Keep the default setting and click on the Next-> Finish button to complete the MySQL package installation.



Step 15: In the next wizard, we can choose to configure the Router. So click on Next->Finish and then click the Next button.

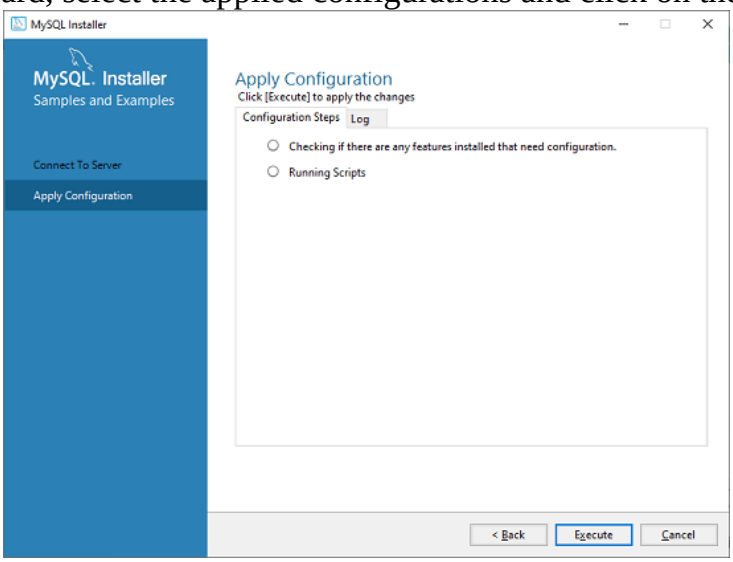


Step 16: In the next wizard, we will see the Connect to Server option. Here, we have to mention the root password, which we had set in the previous steps.

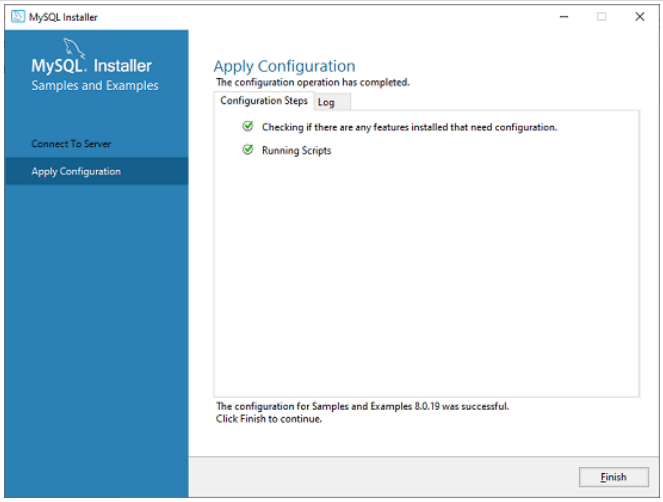


In this screen, it is also required to check about the connection is successful or not by clicking on the Check button. If the connection is successful, click on the Execute button. Now, the configuration is complete, click on Next.

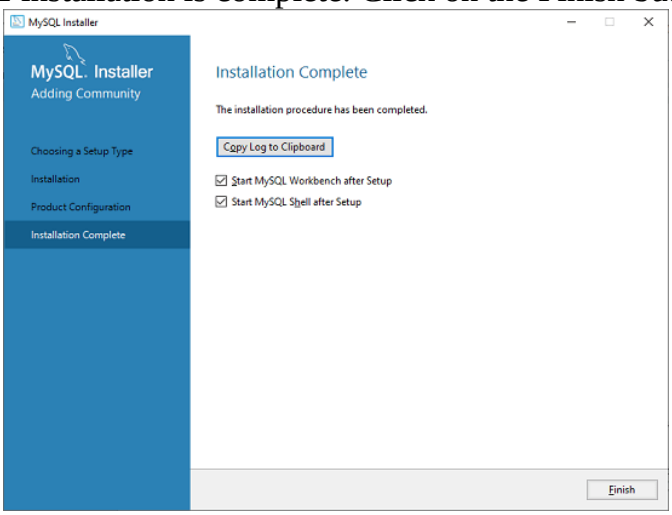
Step 17: In the next wizard, select the applied configurations and click on the Execute button.



Step 18: After completing the above step, we will get the following screen. Here, click on the Finish button.



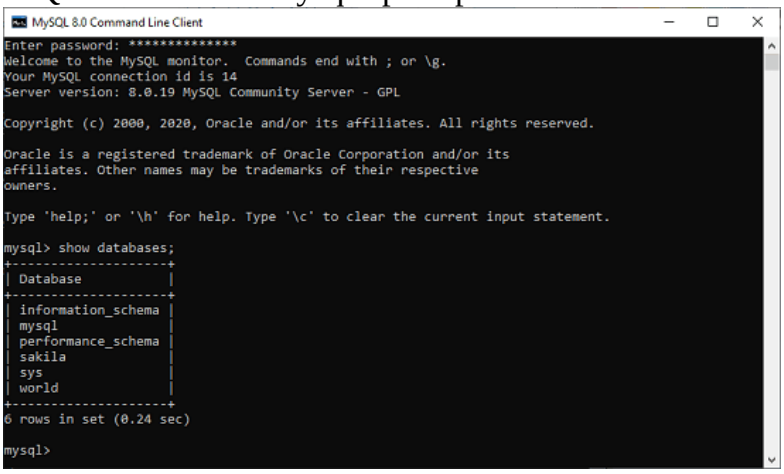
Step 19: Now, the MySQL installation is complete. Click on the Finish button.



Starting MySQL instance

Once MySQL has been successfully installed, the base tables have been initialized, and the server has been started, you can verify its working via some simple tests.

Open your MySQL **Command Line Client**; it should have appeared with a **mysql> prompt**. If you have set any password, write your password here. Now, you are connected to the MySQL server, and you can execute all the SQL command at mysql> prompt as follows:



History of MySQL

MySQL is an open source SQL (structured query language) database management system. It is a system that helps store and manage data efficiently.

- Unireg, which is the code base of MySQL, was started in **1981**.
- MySQL was founded in **1995** in Sweden.
- In **2000**, MySQL went open source, so it could be accessed and used by all.
- In the year **2001**, Marten Mickos was elected as the CEO of MySQL.
- In the year **2002**, MySQL launched its headquarters in USA, in addition to Sweden headquarters.

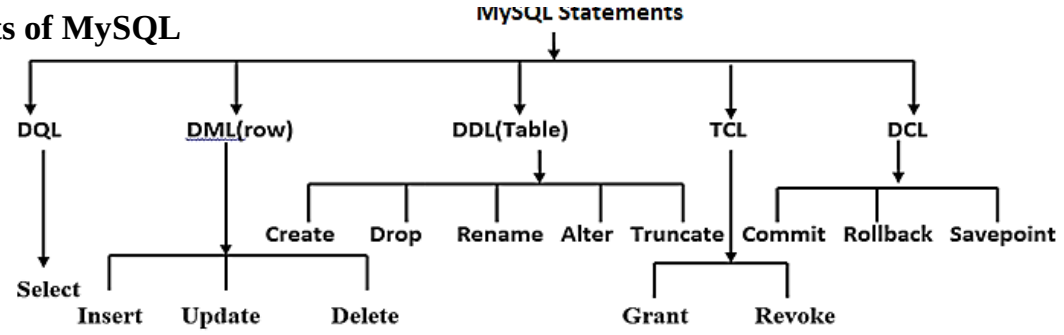
- In **2003**, MySQL entered into a partnership with SAP, and many features were developed in MySQL keeping SAP in mind.
- In **2005**, MySQL launched MySQL Network. Oracle purchased Innobase, which is the backend of MySQL's InnoDB storage.
- In **2008**, MySQL was acquired by Sun Microsystems.
- In **2009**, Sun Microsystems and Oracle entered intoa definitive agreement under which Oracle acquired Sun Microsystems.
- The mSQL database system was designed to connect the data that is stored in tables using customized fast low level (ISAM) routines.
- ISAM refers to indexed sequential access method, which is a file management system. It is a technique that helps access records in the tables sequentially, i.e in the same order in which the records were entered into the table, or randomly with the help of an index. Every index can be used to define a different order for the records in the table.
- ISAM was originally developed by IBM before the development of VSAM (Virtual Storage Access Method) and relational databases.
- When mSQL was tested to see its efficiency, it was observed that mSQL was not quick enough or flexible enough for the requirements in hand.
- MySQL was initially created for personal usage from mSQL based on the low-level language ISAM.
- MySQL has been named after co-founder Monty Widenius's daughter- My. The logo, a dolphin is known as 'Sakila'.

MySQL Features

- **Easy to use:** MySQL is easy to use. We have only basic knowledge of SQL.
- **Secure Data Protection:** Only authorized user's access databases. Passwords are encrypted.
- **Client/ Server Architecture:** MySQL follows the working of a client/server architecture. There is a database server (MySQL) and randomly many clients (application programs), which communicate with the server; that is, they can query data, save changes, etc.
- **Open Source and Free:** MySQL is free to use so that we can download it from MySQL official website without any cost.
- **High Scalable and Flexible:** MySQL supports multi-threading that makes it easily scalable. You can run deeply embedded applications and create data warehouses holding a huge amount of data up to as much as 50 million rows or more. The default file size limit is about 4 GB.
- **Speed:** MySQL is one of the very fast database languages.
- **Compatible on many operating systems:** MySQL is compatible to run on many operating systems like Windows, Linux etc. MySQL also provides a facility that the clients can run on the same computer as the server or on another computer
- **Allows roll-back:** MySQL allows transactions to be rolled back, commit, and crash recovery.
- **Memory efficiency:** Its efficiency is high because it has a very low memory leakage problem.
- **High Performance:** MySQL is faster, more reliable, and cheaper because of its unique storage engine architecture. It provides very high-performance results in comparison to other databases without losing an essential functionality of the software. It has fast loading utilities because of the different cache memory.
- **High Productivity:** MySQL uses Triggers, Stored procedures, and views that allow the developer to give higher productivity.
- **Platform Independent:** It can download, install, and execute on most of the available OS.
- **Partitioning:** improves the performance and provides fast management of the large database.
- **GUI Support:** MySQL provides a unified visual database GUI tool "**MySQL Workbench**" to work with database architects, developers, and Database Administrators. MySQL Workbench provides SQL development, data modeling, data migration, and comprehensive administration tools for server configuration, user administration, backup, and many more. MySQL has a fully GUI supports from MySQL Server version 5.6 and higher.

- **Dual Password Support:** MySQL version 8.0 provides support for dual passwords-one is the current password, and secondary password allows us to transition to the new password.

Components of MySQL



➢ DDL: Data Definition Language

All DDL commands are auto-committed. It saves all the changes permanently in database.

- DDL statements are used for creating, modifying and dropping structure of database objects like table, view, database etc.
- It also defines indexes (keys), specifies links between tables, and forces constraints between tables.

Command	Description
create	Creates a database object like new table, a view of a table etc.
alter	Alters or modifies structure of an existing database object like table, view etc.
truncate	remove all records from a table, including all spaces allocated for the records
drop	Deletes an entire database object like table, view etc.
rename	Rename a database object like table, view etc.

➢ DML: Data Manipulation Language

DML statements used for storing, retrieving, modifying, & deleting data within the database. DML commands are not auto-committed. Changes are not permanent to database, they can be rolled back.

Command	Description
insert	insert data (new row) into a table
update	updates existing data(row) within a table
delete	deletes all records from a table, the space for the records remain

➢ TCL: Transaction Control Language

TCL statements are used to manage the changes made by DML statements. It allows statements to be grouped together into logical transactions. These commands can cancel changes made by other commands by rolling back to original state. It can also make changes permanent.

Command	Description
commit	to permanently save
rollback	to undo change
savepoint	to save temporarily

➢ DCL : Data Control Language

DCL statements control access of data and database. It assign permission to user for use database object. It can also take back (revert) the permission. It allows central administration of user privileges and access.

Command	Description
grant	gives user's access privileges to database
revoke	withdraw access rights given with GRANT command (take back permission)

➢ DQL: Data Query Language

DQL allows getting data from the database. When SELECT is fired against a table or tables, the result is compiled into a temporary table.

Command	Description
select	retrieve records from one or more table

Data types in MySQL

A Data Type specifies a particular type of data, like integer, floating points, Boolean, etc. It also identifies the possible values for that type, the operations that can be performed on that type, and the way the values of that type are stored. In MySQL, each database table has many columns and contains specific data types for each column.

We can determine the data type in MySQL with the following characteristics:

- ✓ The type of values (fixed or variable) it represents.
- ✓ The storage space it takes is based on whether the values are a fixed-length or variable length.
- ✓ Its values can be indexed or not.
- ✓ How MySQL performs a comparison of values of a particular data type.

MySQL supports number of different data types that can be broken into the following categories: numeric, date and time, string types, spatial types, and JSON data types.

➢ Numeric Data Type

These data types can include the exact numeric data types like integer, decimal, numeric, etc. as well as the approximate numeric data types like float, real, and double precision. It also supports bit datatype to store bit values. In MySQL, numeric data types are categories into two types, either signed or unsigned except for bit data type.

Data Type	Description
int	INT is used for storing exact numbers. It is a normal-sized integer that can be signed or unsigned. We can specify a width of up to 11 digits. It requires 4 bytes for storage. There are five INT types- TINYINT, INT, SMALLINT, MEDIUMINT, and BIGINT (the range of TINYINT is the smallest and of BIGINT is the maximum).
float(m,d)	The length of digits is specified using *the m* parameter. The number of digits after the decimal point is specified in the *d* parameter. Decimal precision can go to 24 places for a float type. It requires 2 bytes for storage. Ex: FLOAT (3,2) can store a number 3.12, here the size of the number is 3 and there are 2 digits after the decimal.
double(m,d)	The length of digits is specified using *the m* parameter. The number of digits after the decimal point is specified in the *d* parameter. Decimal precision can go to 53 places for a double. Real is a substitute for double. It requires 8 bytes for storage. Ex: 502.00232
decimal(m,d)	DECIMAL is used for columns that preserve exact precision e.g., money data in accounting systems. The length of digits is specified using *the m* parameter. The number of digits after the decimal point is specified in the *d* parameter. Numeric is a synonym for decimal. Ex: 702. 23
bit(m)	BIT is used to store bit values i.e a number containing 0's and 1's only. The number of bits per value is specified using the *size* parameter. The *size* parameter can hold a value from 1 to 64. The default value for *size* is 1. Ex: 1101
boolean	It is used only for the true and false condition. It considered numeric value 1 as true and 0 as false.

➢ Date and Time Data Type:

This data type is used to represent values such as date, time, datetime, timestamp, and year. Each type contains values, including zero. When we insert the invalid value, MySQL cannot represent it, and then zero value is used.

Data Type	Description
year[(2|4)]	YEAR can be used for storing a year in a four-digit format. Values are allowed in the four-digit format: 1901 to 2155, and 0000. MySQL converts Year values 2 digits to 4 digits. It takes 1 byte for storage. Ex: '70' to '99' to 1970 to 1999.
date	Managing dates. It can store the date in the YYYY-MM-DD format. Dates within the range from '1000-01-01' to '9999-12-31' can be stored. It takes 3 bytes for storage. Example: 2020-10-24
time	TIME is used for storing the time of a day. The time can be stored in the hh:mm:ss format only. You can enter time within the range from '-838:59:59' to '838:59:59'. It takes 3 bytes plus fractional seconds for storage. Example: 09:00:00

datetime	It contains a combination of date and time. The format for storing data and time is YYYY-MM-DD hh:mm: ss. Dates within the range from '1000-01-01 00:00:00' to '9999-12-31 23:59:59' can be stored. You can add DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time. Values range from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. It takes 5 bytes plus fractional seconds for storage. Example: 2020-10-24 11:55:40
timestamp(m)	Values range from '1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' TC. It takes 4 bytes plus fractional seconds for storage. Example: 2020-10-24 11:55:40

➢ String Data Types:

The string data type is used to hold plain text and binary data like files, images, etc. MySQL can perform searching and comparison of string value based on the pattern matching such as LIKE operator, Regular Expressions, etc.

data type	Description
char(size)	It can hold a fixed-length string. Maximum size of 255 characters.
varchar(size)	Stands for VARIABLE CHARACTER. It can hold a variable-length string alphabets, number, or special characters). The range of characters can be between 0 and 65,535.
text(size)	TEXT holds up to 65,535 bytes or 64KB of data. The TEXT is useful for storing long-form text strings such as articles, blogs, etc. It has more features than CHAR and VARCHAR. It can hold from 1 byte to 4 GB of data.
binary(size)	Similar to CHAR(), but stores fixed-length binary byte strings. It does not contain any character set.

➢ Binary Large Object Data Types (BLOB):

BLOB is a **B**inary **L**arge **OB**ject that can hold a variable amount of data. It can store binary data such as images, multimedia, and PDF files.

data type	Description
blob(size)	Maximum size of 65,535 bytes.
longblob	Maximum size of 4gb or 4,294,967,295 bytes.

➢ Spatial Data Types

It is a special kind of data type which is used to hold various geometrical and geographical values. It corresponds to OpenGIS classes. The following table shows all spatial types that support in MySQL

Data Types	Description
geometry	It is a point or aggregate of points that can hold spatial values of any type that has a location.
point	A point in geometry represents a single location. It stores the values of X, Y coordinates.
polygon	It is a planar surface that represents multisided geometry. It can be defined by zero or more interior boundary and only one exterior boundary.
linestring	It is a curve that has one or more point values. If it contains only two points, it always represents Line.
geometrycollection	It is a kind of geometry that has a collection of zero or more geometry values.
multilinestring	It is a multi-curve geometry that has a collection of linestring values.
multipoint	It is a collection of multiple point elements. Here, the point cannot be connected or ordered in any way.
multiplygon	It is a multisurface object that represents a collection of multiple polygon elements. It is a type of two-dimensional geometry.

➢ JSON Data Type

MySQL provides support for native JSON data type from the version v5.7.8. This data type allows us to store and access the JSON document quickly and efficiently. The JSON data type has the following advantages over storing JSON-format strings in a string column:

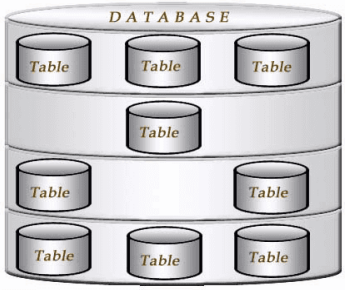
1. It provides automatic validation of JSON documents. If we stored invalid documents in JSON columns, it would produce an error.
2. It provides an optimal storage format.

Creating databases and show databases

A database is used to store the collection of records in an organized form. It allows us to hold the data into tables, rows, columns, and indexes to find the relevant information frequently. We can access and manage the records through the database very easily.

MySQL implements a database as a directory that stores all files in the form of a table. It allows us to create a database mainly in **two ways**:

1. MySQL Command Line Client
2. MySQL Workbench



Syntax: create database [if not exists] database_name;

Ex: create database employee;

➢ Selecting Database

SELECT Database is used in MySQL to select a particular database to work with. This query is used when multiple databases are available with MySQL Server. You can use SQL command **USE** to select a particular database.

Syntax: use database name;

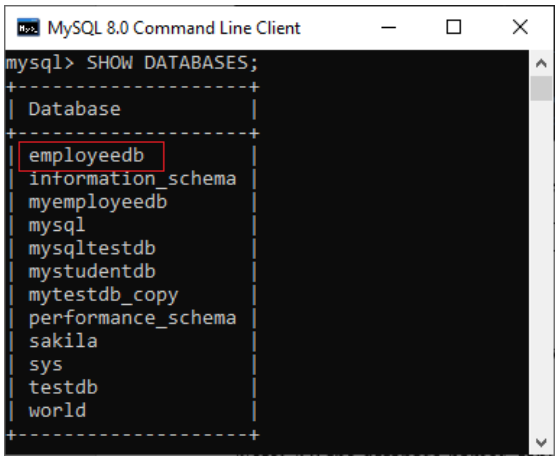
Ex: use employeedb;

➢ Show/List Databases

SHOW statement displays a list of currently existing databases on the server.

Syntax: Show databases;

Ex:



➢ Drop Database

We can drop/delete/remove a MySQL database quickly with the MySQL DROP DATABASE command. It will delete the database along with all the tables, indexes, and constraints permanently. Therefore, we should have to be very careful while removing the database in MySQL because we will lose all the data available in the database. If the database is not available in the MySQL server, the DROP DATABASE statement throws an error.

Syntax: drop database [if not exists] database_name;

Ex: drop database employeedb;

➢ MySQL show tables statement

MySQL 'show tables' statement displays a list of the tables in the database in use. If there is no table in the database, it returns empty rows.

Syntax: Show tables;

➢ COPY Database

MySQL copy or clone database is a feature that allows us to create a duplicate copy of an existing database, including the table structure, indexes, constraints, default values, etc. Making a duplicate copy of an original database into a new database is very useful when accidentally our database is lost or failure. The most common use of making a duplicate copy of the database is for data backups. It is also useful when planning the major changes to the structure of the original database.

*In MySQL, making the clone of an original database is a **three-step process:***

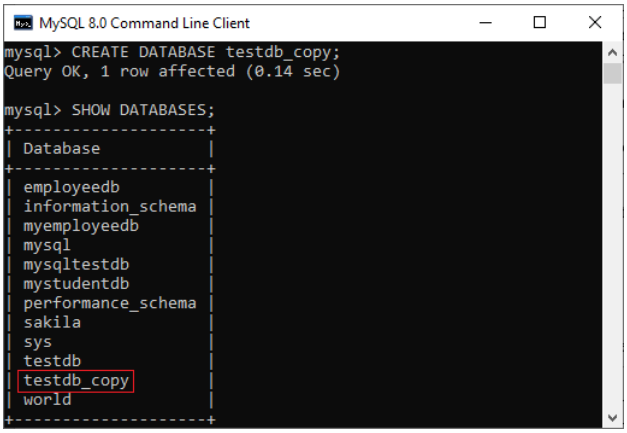
1) First, the original database records are dumped (copied) to a temporary file that holds the SQL commands for reinserting the data into the new database.
2) Second, it is required to create a new database.
3) Finally, the SQL file is processed, and the data will be copied into the new database.

We need to follow these steps to copy a database to another database:

1. First, use the **CREATE DATABASE** statement to create a new database.
2. Second, store the data to an **SQL file**. We can give any name to this file with a **.sql** extension.
3. Third, export all the database objects along with its data to copy using the **mysqldump** tool and then import this file into the new database.

Example: we will copy the **testdb** database to **testdb_copy** database using the following steps-

```
create database testdb_copy;
show databases;
```



Now, open a DOS terminal window to access the MySQL server on the command line. For example, if we have installed the MySQL in the **C folder**, copy the following folder and paste it in our DOS command. Then, press the **Enter** key.

```
C:\Users\ecs> CD C:\Program Files\MySQL\MySQL Server 8.0\bin
```

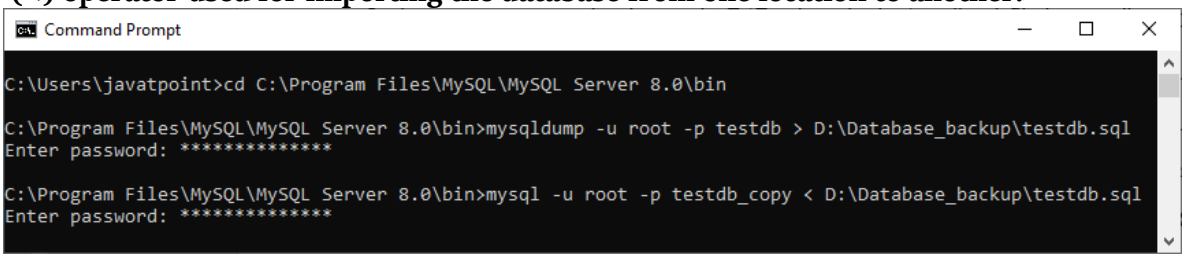
In the next step, we need to use the **mysqldump** tool to copy the database objects and data into the SQL file. Suppose we want to dump (copy) the database objects and data of the testdb into an SQL file located at **D:\Database_backup folder.**

```
mysqldump -u root -p testdb > D:\Database_backup\testdb.sql
Enter password: **********
```

The above statement exports the database objects and data of the testdb database to **D:\Database_backup\testdb.sql (>)operator used for exporting the database from one location to another.** In the next step, we need to import the D:\Database_backup\testdb.sql file into testdb_copy database.

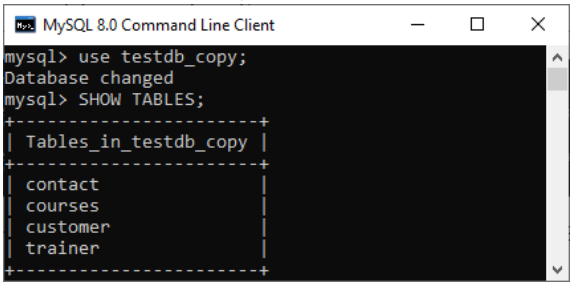
```
mysql -u root -p testdb_copy < D:\Database_backup\testdb.sql
Enter password: **********
```

(<) operator used for importing the database from one location to another.



Finally, we can verify whether the above operation is successful or not by using the **SHOW TABLES** command.

```
SHOW TABLES;
```



all the objects and data from the testdb database to testdb_copy database have successfully copied.

Performing Operation on Table Data

- ♣Creating table, creating a table from a table,
- ♣Populating tables with data, inserting data into a table from another table
- ♣ Retrieving data from tables
- ♣Sorting data in a table
- ♣Deleting data from table
- ♣ Updating data in tables
- ♣ Altering table- Adding new columns, dropping columns, Modifying existing columns, Rename existing column, Renaming table, Changing a table type
- ♣ Displaying a table structure
- ♣Finding out the tables created by user

Create Table

CREATE TABLE statements used for define the structure of a table. A table is used to organize data in the form of rows and columns. It is used for storing and displaying records. In a table structure you should define various fields, their data types and constraints. Each column uniquely defined in the table. Each column has a minimum of three attributes, a name, data type and size (i.e. column width). Each column definition is separated from other column definition by a comma (,).

Rules for Creating Tables

1. A name of the table can have maximum up to 30 characters
2. Alphabets from A-Z, a-z and numbers from 0-9 are allowed
3. Name should start with an alphabet
4. A special characters is allowed (Special characters like _, $, # etc.).
5. SQL word is reserved and you cannot use. For example: create, select, and so on.

MySQL allows us to create a table into the database mainly in **two ways**:

1. MySQL Command Line Client
2. MySQL Workbench

MySQL allows us to create a table into the database by using the **CREATE TABLE** command.

Syntax: create table [if not exists] table_name (column_definition1, column_definition2, ..., table_constraints);	**Example:** create table emp(id int , name varchar(30), occupation varchar(25), age int);

- **To see the newly created table:** show tables;
- **See the table structure:** describe emp;
- **Creating a Table from Previously Created Table**

Syntax: create table <tablename1><columnname1>,< columnname2>... <columnname n>) as select <columnname1>,<columnname2>...<columnnamen> from <tablename2> where <condition>;

If the 'TableName2' is having records then the target table 'TableName1' will be occupied with the same records.

Example

1. create table emp1 as select * from emp;
2. To Change column name at table creation time. For example-CURBAL to BALANCE.

create table emp2 (eno, enmae, eaddress) as select eno,enmae,eadd from emp;

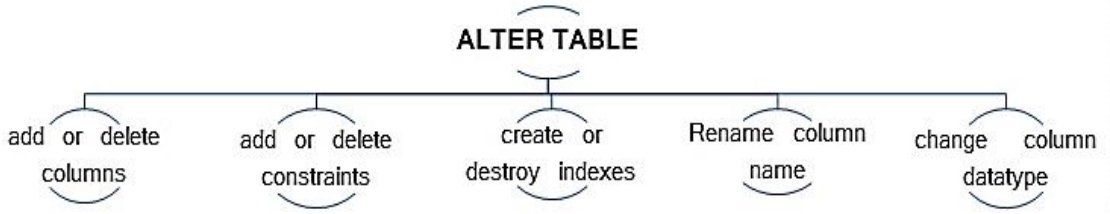
- **Create Table without Records (AS SELECT statement)**

To create a new table without records from the existing table (i.e. create structure only).AS SELECT statement used with WHERE clause condition that cannot be satisfied.(i.e. It will just retrieve the table structure not records. Thus, new table will be created empty.)

Syntax	Example
create table <new table name> as select * from <old tablename> where 1=2;	create table emp3(eno, enmae, eaddress) as select eno, enmae, eadd from empwhere eno=1;

Alter Table (Modifying structure of Table)

The ALTER TABLE statement is used to change the structure of an existing table. It is used to add, modify or delete columns an existing table. It is also used to rename a table name and column names. You can also add and drop various constraints on an existing table.



Restrictions on the ALTER table

1) User cannot change the column name
2) User cannot decrease the size of column if record in the column.

Syntax: alter table table_name ADD new_column_name column_definition [FIRST| AFTER column_name];

Adding New Columns

- Add single column in table

Syntax: alter table table_name add (column_name <datatype>(size));

Example: alter table emp add (city varchar(20));

- Add multiple columns in table

Syntax: alter table table_name add (column1 column_definition, column2column_definition,... column_n column-definition);

Example: alter table emp add (email_id varchar (20), gender char(6));

- Add multiple columns in table with column position.

alter table emp add salary int(10) after name, add age int after gender age ;

Dropping a Column: remove column from the table.

Syntax: alter table <table_name> drop column <column_name>;

Example: alter table emp drop column city;

Modifying Existing Columns

- Modify an existing column in table

Syntax: alter table table_name modify (<column_name><new data_type><size>);

Example: alter table emp modify (ename char(40));

- Modify multiple columns in table

Syntax: alter table table_name modify (<column_name1><new data_type><size>, <column_name2><new data_type><size> ... <column_name n><new datatype><size>);

Example: alter table emp modify (email_id varchar(30), eadd varchar(25));

Rename Column in table: rename column name

Syntax: alter table table_name rename column <old column_name> to <new column_name>;

Example: alter table emp change column eadd to empaddress;

Rename table: rename column name

Syntax: alter table table_name rename to <new column_name>; **or**
rename <old table_name> to <new table_name>;

Example: alter table emp rename to workers; **or**
rename emp to workers;

Alter Table To Add / Remove Constraints

- To add a primary key/Composite Primary Key constraint after creation of a table

Syntax: alter table <table_name> add <constraints_name> (columns);

Example: alter table emp add primary key (eno);
alter table emp add primary key (id, name); //Composite Primary Key

alter table emp modify salary decimal(18, 2) not null;

- To add a foreign key constraint after creation of a table

  Example: alter table emp add foreign key (deptno) references dept (deptno);

- To remove primary key constraint

  Syntax: alter table <table_name> drop <constraints name>;

  Example: alter table emp drop primary key;

  It will remove primary key constraint from a table.

- To add user defined constraint

  Example: alter table emp add constraint myuniqconstraint unique(age);

- To remove user defined constraint

  Syntax: alter table <table_name> drop constraints <const_name>

  Example: alter table emp drop constraint myuniqconstraint;

  It will remove the constraint, which was defined on the table.

Drop Table (Drop a table from the database)

Drop command is used to remove any object such as table, index, view, package and function, all data, indexes, triggers, constraints and permission specifications. If a table is dropped, all records within table are lost and cannot be recovered. Use very careful this command.

Syntax: drop <object><object_name>; or drop database <Database_Name>;

Example: drop table emp; or drop view vw_empdept;

Note: 1.DROP statement removes the table.
2. It deletes all its data.
3. It deletes constraints and indexes associated with it.
4. Any views and synonyms are remain but are invalid.
5. Any pending Transaction is committed.
6. Only the creator (user) of the table DROP ANY TABLE privilege can remove a table.

Truncate Statement

It is used to delete the entire table data permanently and free the containing space. But table structure will be remains present but records cannot be ROLLBACK.

Syntax: truncate table <table_name>;

Example: truncate table emp;

- **Difference b/w DROP and TRUNCATE statements**

When you drop a table:

- Table structure will be dropped
- Relationship and Integrity constraints will be dropped
- Access privileges will also be dropped

When we TRUNCATE a table, the table structure remains the same.

- Truncate operations drop and recreate table faster than deleting rows one by one.
- It is used to delete all the rows from the table and free the containing space.

DESC (View Structure of table) : It used to view structure of a table.

Syntax: describe <Table_Name> OR desc <Table_Name >

Example: desc emp;

- **Difference between Drop, Delete and Truncate**

Drop	Delete	Truncate
1. Whole structure and data will drop. 2.can't Rollback 3.can't use where clause 4.All data and structure drop simultaneously	1. Structure will not drop, data will drop. 2.can Rollback 3.can use WHERE clause 4.Row by Row data Delete	1. Structure will not drop, all data will drop. 2.can't Rollback 3.can't use WHERE clause 4. Truncate operation drop and recreate the table.

Insert Statement

INSERT statement is used to store or add data in MySQL table within the database. We can perform insertion of records in two ways:

1. Insert record in a single row
2. Insert record in multiple rows

1. Single Row:

```
Syntax:insert into table_name (field1, field2,.....fieldn )  values ( value1, value2,.....valuen );
Example:insert into emp (id, name, occupation, age) values (101, 'Ravi', 'engineer', 32);
```

2. Multiple Rows:

```
Syntax:insert into table_name values (val1, val2,...valn ), (val1, val2,...valn).....(val1,val2,...valn);

Ex:insert into emp values (102, 'Joseph', 'Developer', 30), (103, 'Mahi', 'Leader', 28), (104,'Sam','Scientist', 45);
```

3. If we want to store records without giving all fields, we use the following partial field statements

```
insert into emp (name, occupation) values ('satish', 'scientist'), ('binod', 'actor');
```

Insert into Select Statement

We want to insert data of one table to other table in the same or different database.It is not easy to enter these data using the insert query manually. We use of **insert into select** query. It allows us to fill tables quickly.**<u>Query copies data from one table and inserts them in the other table</u>**.

- The data types in source and target tables must be the same.
- The existing records in the target table should be unaffected.

- Adding single or multiple from one table into another table

```
Syntax: insert into table_name2 select * from table_name1;  or    insert into table2 table table1;
        insert into person select * from emp;            or insert into person table emp;
```

- Copy only specific records from one table into another table using 'where clause'

```
Syntax: insert into table_name2 select * from table_name1  where condition;

        insert into person select * from empwhereoccupation='Developer';
```

SELECT (Retrieving Data Using Query)

SELECT Statement used to fetch data from database table which returns data in the result sets of table. SELECT command is used to retrieve rows selected from one or more tables.

➢ **All Rows and All Columns**

```
Syntax:  select * from <Table_Name>;
Example: select * from emp;
```

It will display data with all rows and columns from the table.

➢ **Filtering Table Data**

When we retrieves specific rows and columns data from a table. SQL provides a method of filtering table data with following ways:

1. Selected columns and all rows
2. Selected rows and all columns
3. Selected columns and selected rows

1. Selected Columns and All Rows

It will display all records with specific fields that user has specified with SELECTcommand.

```
Syntax: select <col_name1>,<col_name2>,.....<col_name n> from < table_name >;
Example: select eno, ename from emp;
```

2. Selected Rows and All Columns

```
Syntax: select * from <table_name> where <condition>;
Example: select * from emp where eno=10;
```

displays only those records that satisfy the specified condition after WHERE clause.

- **Where clause** applies a filter on the retrieved rows. When a WHERE clause is added to the SQL query, the Oracle engine compares each record in the table with specified condition. All operators such as logical, arithmetic, etc. can be used in the condition of WHERE clause.

3. Selected Columns and Selected Rows

Syntax: select <col_name1>,<col_name2>,.<col_name n> from <table_name> where <condition> ;

Example: select eno, ename from emp where eno=10;

It will display all records with specific fields after specified condition WHERE will be true.

- **Eliminating Duplicate Rows**

In a table, there may be a chance to exist a duplicate value. DISTINCT command is used to retrieve only unique data. SELECT UNIQUE & SELECT DISTINCT statements are same.

Syntax: select distinct <Col_Name1>,<Col_Name2>,...<Col_Name n> From <Table_name>;

or

select distinct * from <table_name>;

Example: select distinct eno, ename from emp;

or

select distinct * from emp;

- **Distinct clause-** allows removing duplicates data from the result set. This clause only used with SELECT statement. It scans all the values of the specified columns and display only unique values. It eliminates rows that have exactly same contents in each column.

- **Arithmetic calculation**

Example: Select employee name, salary and compute salary * 0.05 for each row retrieved:

Select ename, sal, sal * 0.05 "Salary" from emp;

Here the default output column sal∗0.05 is renamed with Salary.

Delete (Deleting Data)

- Delete statement is used to delete one or more rows from a table.
- Delete query use following two way,
  - Remove all rows
  - Remove only specific rows
- Delete statement does not free containing space by the table.

Syntax: delete from <table_name> where <condition>;

- If condition is not specified then it will be delete all rows.

Example: delete from emp; // it will delete all the records from a emp table.

- If condition is specified then it will delete all rows that satisfy the specified condition.

Example: delete from emp where eno>10;

It will delete all the records for which ENO is greater than 10.

UPDATE (Updating a Contents of Table)

The UPDATE statement changes in existing rows data either adding new data or modifying existing data.User can update

- all rows from table (update all table rows)
- selected rows from table (update only specific data/rows)

Syntax: update <table_name> set <columnname1>=<expression1>, <columnname2>= <expression2>....., <columnnamen>=<expressionn> where <condtion>;

OR

update <table_name> set <column_name=expression> where <conditions>;

Example: update emp set city='sangola';

- **All rows from table:** If condition is not specified then it will be update entire table rows.It can update multiple fields at the same time.

  update emp set salary=500;

```
update emp set address='pune', salary=1000;
```

- **Selected rows from table:** If condition is specified then it will be update only specific table rows.It can update single fields at the same time.

```
update emp set city='sangola' where eno=10;
```

MySQL clauses, Function, Operators

♣MySQL clause-where distinct, from, order by, group by, having, rollup

♣ MySQL Function-Aggregate functions, Math functions, String functions, Date and Time functions, control flow functions and expressions, comparison functions, Window functions

♣ MySQL operators- and, or, and or, like, in, not, is null, is not null, between, comparison operator

---

MySQL operators

The symbols which are used to perform logical and mathematical operations in MySQL are called MySQL operators. MySQL reserved words or characters are known as operators.

Three types of operators in MySQL:

1. MySQL Arithmetic Operators(+,-,/,*)
2. MySQL Comparison Operators (<,>,=,!=,<=,>=,==)
3. MySQL Logical Operators (and, or)

MySQL Arithmetic Operators:

Arithmetic operators in SQL are used to perform mathematical calculations like addition,subtraction, multiplication, division and modulus in statements.

DUAL table: It is a system defined table which contains only one column to perform calculations on users own data.

Arithmetic Operators	Example
+ (Addition)	Select 10+2 from dual;
– (Subtraction)	Select 10-2 from dual;
* (multiplication)	Select 10*2 from dual;
/ (Division)	Select 10/2 from dual;
% (Modulus)	Select 10%2 from dual;

```
select (90000*10)/100 "10% of 90000" from dual;
select sal " Basic Sal", (0.02*sal) " TA" from emp;
```

MySQL Comparison Operators: used to compare values by specifying conditions on the columns.

Operators	Example/Description
>	x > y (x is greater than y)
<	x < y (x is less than y)
>=	x >= y (x is greater than or equal to y)
<=	x <= y (x is less than or equal to y)
=	x == y (x is equal to y)
!= or <>	x != y or x <> y (x is not equal to y)
!<	x !< y (x is not less than y)
!>	x !> y (x is not greater than y)

MySQL Logical Operators:

Logical operators in SQL are used to perform logical operations on the given expressions in SQL statements. There are many operators in SQL which are used in SQL statements in the WHERE clause. They are

Operator	Description
AND	Display output if all conditions are true. If any one condition was failed then will not display O/P.
OR	Display output if anyone condition is true. If all conditions are false then it will not display output.
BETWEEN	The **BETWEEN** operator is used to get values within range.
IN	The **IN** operator is used to search for specified value matches any value in set of multiple values.
ALL	The **ALL** operator returns true when value matches with all values in single column set of values. It's like **AND operator** it will compare value against with all values in column.

ANY	The **Any** operator in SQL returns true when value matches with any value in single column set of values. It's like **OR operator** it will compare value against with any value in column.
LIKE	The **LIKE** operator is used to search for character string with specified pattern using wildcards in column.
EXISTS	The **EXISTS** operator is used to show result if subquery returns data.
NOT	The **NOT** operator is a negate operator that means it will show data for opposite of conditions what we mentioned in SQL statement.
SOME	The **SOME** operator is used to compare value with a single column set of values returned by subquery. **SOME** must match at least one value in subquery and that value must be preceded by comparison operators.

Comparison Operators =, >, <, >=, <=, !=, <>

Example:

```
select * from student where no = 2;
select * from student where no < 4;
select * from student where no > 3;
select * from student where no <= 7;
select * from student where no >= 3;
select * from student where no != 2;
select * from student where no <> 2;
```

- **AND Operator:** returns output when all the conditions become true.

Syntax: select * from <table_name> where <condition1> and <condi2>and<condition n>;
Example: select * from student where no=2 **and** marks>=200;

- **OR Operator:** returns output when either one of the conditions becomes true.

Syntax: select * from <table_name> where <condition1>and<condition2> or <conditionn>;
Example: select * from student where no=2 **or** marks>=200;

- **NOT** will negate the condition. It will select the records where condition is not satisfied.
- **Range Searching:** Three types for searching data in a range.
  1. BETWEEN
  2. IN
  3. IS

Between Operator

- Between operator is used to select data that is within a range of values.
- Between operator that contain specified lower and upper limit values.
- Two values in the range must be linked with keyword AND.
- Between operators can be used with numeric and character data types.

Syntax: select * from <table_name> where <col> between <lower bound> and <upper bound>;
Example: select * from student where marks between 200 **and** 400;
Select empno, ename, deptno from emp where deptno>=15 **and** deptno<=25;

Not Between Operator: used for to select data which is not within the range.

Syntax: select * from <table_name> where <col> **not between** <lower bound> **and** <upper bound>;
Example: select * from student where marks **not between** 200 **and** 400;
select * from emp where empno **not between** 10 **and** 20;

In Operator

- If a value is required to be compared with list of values then IN predicate is used.
- IN operator reduces the need to use multiple OR conditions.
- Arithmetic operator (=) compares a single value to another single value.

Syntax: select * from <table_name> where <col> in (value1, value2, value3...valuen);
Example: select * from student where no **in** (1, 2, 3);
select empno, ename, deptno from emp where deptno 10 **or** deptno=20;

```
select   empno, ename, deptno from emp where deptno in (20,30);
```

- **Not In Operator:** used to select all the rows where value does not match in the list of values specified

```
Syntax: select * from <table_name> where <col> not in(value1, value2,value3...value n);
Example: select * from student where no not in (1, 2, 3);
         Select   empno, ename, deptno from emp where deptno  not in (20,30);
```

- **Is Null Operator:** used to check the column value is null or not, if it is null display output

```
Syntax: select * from <table_name> where <col> is null;
Example:  select * from student where marks is null;
```

1) If expression is a NULL value, the condition evaluates to TRUE.
2) If expression is not a NULL value, the condition evaluates to FALSE.

- **Is Not Null Operator:** used to test for NOT NULL value in the specified column.

```
Syntax:  select * from <table_name> where <col> is not null;
Example:  select * from student where marks is not null;
```

- **Like Operator**

Like operator is used to retrieve data based on specify pattern or characters. Like operator selects those rows only containing fields that match specified portions of character strings. LIKE is used with char, varchar, text, date time and small date time data. A wildcard allows the user to match fields that contain certain letters.

```
Syntax: select * from <table_name> where <col> like <pattern>;
```

%	It matches zero or more any characters.
_	Any single character search with the specified pattern
[ ]	Any single character within the specified range e.g.([a-f]) or set [abcdef])

Example:

```
Create table customer ( cno int,
cname varchar(10),
balance int(6)
city varchar(10));
```

Cno	Cname	Balance	City
1	Arjun	12550	Pandharpur
2	Ramesh	16550	Solapur
3	Jagan	18000	Sangola
4	Madhu	19000	Kolhapur
5	Varun	15250	Sangli
6	Ritu	14850	Satara

Find customer names whose salary is 15000.

```
select * from customer where salary like 15000;
```

Find customer names whose city name beginning start with 'Sa'.

```
select cname from customer where city like 'Sa%';
```

Find customer names ending with 'un'.

```
select * from customer where cname like '%un';
```

Find customer name whose name's second letter start with 'a'.

```
select * from customer where cname like '_a%';
```

Display customer details whose balance's third number start with '5'.

```
select * from customer where balance like '__5%';
```

Display customer details whose name's third letter start with 'h' from ending.

```
select * from customer where cname like '%_t%';
```

Find customer names whose city name start with "S" and end with "a"

```
Select cname from customer where city like 'S%a';
```

Find customer names whose name contains 2 a's.

```
select * from customer where cname like '%a%a%';
```

Find customer name whose city name have "ang" in any position

```
select  cname from customer where city like '%ang%';
```

MySQL Functions

MySQL functions can be used to manipulate data and returning the result. A function is performs a specific task. A function can accept user-supplied variables or constants and perform operation on them. Such variables / constants are known as *arguments.*

Function_Name [(argument-1, argument-2,.)]

Functions can be classified according to different data types are:

1) Scalar Function (Single Row Function)
2) Aggregate (Group Function)
3) Math functions
4) String functions (For String data type)
5) Date and Time functions For Date data type)
6) Control flow functions and expressions
7) Comparison functions
8) Window functions

Single Row Functions

- Functions that work on only one value at a time are called SCALAR Functions.
- A single row function returns one result for every row of table / view.
  Example: LENGTH function calculates length of one string value.
- Single row functions can be grouped together depending on the data type of their arguments and return values. **Ex:** LENGTH function is related to string data type.

Aggregate functions

- Aggregate / Group Functions performs a calculation on multiple values and returns a single value.
  Example: SUM function calculates total of a column.
- Group function returns a single result row for a group of rows.
- Aggregate functions can appear in SELECT lists and in ORDER BY and HAVING clauses.
  They are commonly used with the GROUP BY clause in a SELECT statement.

Function	Description
AVG	Returns the average value of the given column or expression **Ex:** select avg (salary) as 'average salary' from emp;
COUNT	Number of rows where the value of the column is not NULL **Ex:** select count (dno) as 'number of deptwise ' from emp;
COUNT(*)	Number of rows returned including duplicates and NULLs **Ex:** select count (*) from employees where eno = 5;
MAX	Return Maximum value of the given column. With character columns, MAX finds the highest value in the collating sequence. MAX ignores any null values. **Ex:** select max(salary) from emp; or select max(ename) from emp;
MIN	Return Minimum value of the given column. With character columns, MIN finds the value that is lowest in the sort sequence. MIN ignores any null values. **Ex:** select min (salary) from emp; or select max(ename) from emp;
SUM	Returns the sum of all the values given column or expression. SUM can be used with numeric columns only. **Ex:** select sum (salary) as 'total salary' from emp;

Math (Numeric) Functions (For Number data type)

MySQL provides a wide range of functions which serves a wide range of arithmetical purposes.

♦ **abs:** returns the absolute positive value of the specified number.

Syntax: abs(*value*)

Ex: select abs(5), abs(-5), abs(0), abs(null) from dual;

```
abs(5)      abs(-5)     abs(0)      abs(null)
----------  ----------  ----------  ------------
5            5          0           NULL
```

♦ **sign:** is used to get a sign of the specified number.

Syntax: sign (value)

Ex: select sign(5), sign(-5), sign(0), sign(null) from dual;

sign(5)	sign(-5)	sign(0)	sign(null)
1	-1	0	NULL

♦ **sqrt:** This will give the square root of the given value.

Syntax: sqrt (value) -- here value must be positive.

Ex: select sqrt(4), sqrt(0), sqrt(null), sqrt(1) from dual;

sqrt(4)	sqrt(0)	sqrt(null)	sqrt(1)
2	0	NULL	1

♦ **mod:** Display remainder value after value divides divisor.

Syntax: mod (value, divisor)

Ex: select mod(7,4), mod(1,5), mod(null,null), mod(0,0), mod(-7,4) from dual;

mod(7,4)	mod(1,5)	mod(null,null)	mod(0,0)	mod(-7,4)
3	1	NULL	0	-3

♦ **power:** Display m power nth value.

Syntax: power(m,n)

Ex: select power(2,5), power(0,0), power(null,null), power(2,-5) from dual;

power(2,5)	power(0,0)	power(null,null)	power(2,-5)
32	1	NULL	0 .03125

♦ **exp:** This will raise e value to the give power.

Syntax: exp (*value*)

Ex: select exp(1), exp(2), exp(0), exp(null), exp(-2) from dual;

exp(1)	exp(2)	exp(0)	exp(null)	exp(-2)
2.71828183	7.3890561	1	NULL	0 .135335283

♦ **ceil:** Display lowest integer value which is greater than or equal to given value.

Syntax: ceil (*value*)

Ex: select ceil(5), ceil(5.1), ceil(-5), ceil(-5.1), ceil(0), ceil(null) from dual;

ceil(5)	ceil(5.1)	ceil(-5)	ceil(-5.1)	ceil(0)	ceil(null)
5	6	-5	-5	0	NULL

♦ **floor:** Display highest integer value which is lessthan or equal to given value.

Syntax: floor (*value*)

Ex: select floor(5), floor(5.1), floor(-5), floor(-5.1), floor(0), floor(null) from dual;

floor(5)	floor(5.1)	floor(-5)	floor(-5.1)	floor(0)	floor(null)
5	5	-5	-6	0	NULL

♦ **round:** Display value *"m"* which is rounded to the *"n"* number of decimal places

Syntax: round (m,n)

Ex: select round(123.2345), round(123.2345,2), round(123.2354,2) from dual;

round(123.2345)	round(123.2345,0)	round(123.2345,2)	round(123.2345,-2)
123	123	123.23	100

♦ **truncate:** Display value **m** which is truncated to the **n** number of decimal places.

Syntax: truncate (m,n)

Ex: select trunc(123.2345), trunc(123.2345,2), trunc(123.2354,2) from dual;

```
truncate(123.2345)     truncate(123.2345,2)     truncate(123.2345,-1)     truncate(123.2345,-2)
--------------         ----------------         -----------------         --------------
123                    123.23                   120                       100
```

♦ **greatest:** Display maximum value from the given values or expressions.

Syntax: greatest (*value1, value2, value3 ... value n*)

Ex: select greatest (1, 2, 3), greatest (-1, -2, -3) from dual;

```
greatest (1,2,3)     greatest(-1,-2,-3)
--------------------  ----------------------
3                     -1
```

♦ **least:** Display minimum value from the given values or expression results.

Syntax: least (*value1, value2, value3 ... value n*)

Ex: select least(1, 2, 3), least(-1, -2, -3) from dual;

```
least(1,2,3)         least(-1,-2,-3)
--------------------  ----------------------
1                     -3
```

String Functions

♦ **upper / ucase:** converts all the characters in a string to uppercase characters.

Syntax: upper (*string*) or ucase(string)

Ex: select upper('computer') from dual; or select ucase('computer') from dual;

```
UPPER
-----------
COMPUTER
```

♦ **lower / lcase:** converts all the characters in a string to lower case characters.

Syntax: lower (*string*) or lcase(string)

Ex: select lower('COMPUTER') from dual; or select lcase('COMPUTER') from dual;

```
LOWER
-----------
computer
```

♦ **length:** returns the length of the input string. The function accepts a string or a character as an input and returns the integer value.

Syntax: length (*string*)

Ex: select length('computer') from dual; or select ename, length(ename) from emp;

```
LENGTH
---------
8
```

♦ **char_length():**used to get the length of the string. The function accepts a string as an input and returns the integer value. This function considered space between two strings as a character

Syntax: length (*string*)

Ex: select char_length('computer science') from dual;

```
LENGTH
---------
16
```

♦ **rpad:** Display given string along with specific char in the right of string.

Syntax: rpad (*string, length [, padding_char]*)

Ex: select rpad('computer',15,'*'), rpad('computer',15,'*#') from dual;

```
RPAD('COMPUTER'        RPAD('COMPUTER'
----------------       ------------------
computer*******        computer*#*#*#*
```

-- Default padding character was blank space.

♦ **lpad:** Display given string along with the specific char in the left of the string.

Syntax: lpad (*string, length [, padding_char]*)

Ex: select lpad('computer',15,'*'), lpad('computer',15,'*#') from dual;

```
LPAD('COMPUTER'          LPAD('COMPUTER'
    ----------------     ----------------
    *******computer          *#*#*#*computer
-- Default padding character was blank space.
```

♦ **ltrim:** removing blank spaces from the left of string.

Syntax: ltrim (*string [,unwanted_chars]*)

Ex: select ltrim(' computer') from dual;

```
LTRIM('  computer')
---------
computer
```

♦ **rtrim:** removing blank spaces from right of string.

Syntax: rtrim (*string [, unwanted_chars]*)

Ex: select rtrim('computer ') from dual;

```
RTRIM('computer  ')
--------
computer
```

♦ **trim:** Display given string by eliminating blank spaces before and after the string.

Syntax: trim (*unwanted_chars*from *string*)

Ex: select trim('i' from 'indiani') from dual;

```
TRIM(
-----
ndian
```

♦ **left():** returns a specified number of characters from the left of a given string.

♦ **right():**returns a specified number of characters from the rightt of a given string.

Syntax: left (string, length) or right (string, length)

Ex: select left('sangola', 3); or select right('sangola', 3);

```
Left('sangola',3)          right('sangola',3)
-----------------          -----------------
San                         ola
```

♦ **replace:** replaces all the occurrences of a substring(new char) within a string (old char).

Syntax: replace (*string, old_chars [, new_chars]*)

Ex: select replace('india','in','xy'), replace('india','in') from dual;

```
REPLACE    REPLACE
-------    -------
Xydia      dia
```

♦ **reverse():** Returns a given string with the order of the characters reversed.

Syntax: reverse(str)

Ex: select reverse('sangola') from dual;

```
reverse('sangola')
-----------------
alognas
```

♦ **concat:** used to combine two strings only.

Syntax: concat (*string1, string2*)

Ex: select concat('computer',' operator') from dual;

```
CONCAT('COMPUTER'
```

```
------------------------
computer operator
```

To combine more than two strings using concatenation operator (||).

```
select 'how' || ' are' || ' you' from dual;
'HOW'||'ARE
---------------
how are you
```

♦ **ascii:** returns the ASCII value for the specific character.

Syntax: ascii (*string*)

Ex: select ascii('a'), ascii('A') from dual;

```
ASCII('A')      ASCII('A')
--------        -----------
97              65
```

♦ **char:** return the character having ASCII value.

Syntax: char (number)

Ex: select char(97) from dual;

```
CHAR
-----
a
```

♦ **substr:** This will be used to extract substrings.

Syntax: substr (string, start_chr_no [no_of_chars])

Ex: select substr('computer',2), substr('computer',2,5), substr('computer',3,7) from dual;

```
SUBSTR(      SUBSTR      SUBSTR
----------   -------     --------
omputer      omput       mputer
```

♦ **instr:** returns position of the first occurrence of the substring within the string **OR**
position: returns the position of a substring within a string

Syntax: instr (str,substr) or position(substr IN str)

Ex: select position('s' in 'sangola'); OR select instr ('sangola', 's');

```
position('s' in 'sangola')        instr ('sangola', 's');
---------------------------       ---------------------------
                1                          1
```

♦ **locate():** returns the position of the first occurrence of a string within a string. If this position is not mentioned, searching starts from the beginning.

Syntax: locate(substring, string, position)

Ex: select locate('st','myteststring');

```
locate('st','myteststring')
-------------------------
5
```

♦ **mid():** extracts a substring from a string. The actual string, position to start extraction and length of the extracted string - all are specified as arguments.

Syntax: mid(string, position, length)

Ex: select mid('myteststring', 4, 3);

```
locate('myteststring',4,3)
-------------------------
est
```

Date Functions

MySQL DATE is a temporal data type for accessing and setting dates by applications. It accepts DATE values only in YYYY-MM-DD format. And MySQL doesn't allow to change it.

MySQL for DATE and TIME functions and storing the values:

- DATE: YYYY-MM-DD
- DATETIME: YYYY-MM-DD HH:MM:SS
- TIMESTAMP: YYYY-MM-DD HH:MM:SS
- YEAR: YYYY or YY

♦ **curdate:** This function returns the current date.

```
Ex:  select curdate();          or  select current_date();
     | CURDATE()  |
     +------------+
     | 2017-07-13
```

♦ **curtime:** This will return the current time in 'HH:SS:MM' format.

```
Ex:  select curtime();   or  select current_time();
     | CURRENT_TIME |
     +--------------+
     | 11:35:45     |
```

♦ **current_timestamp:** This function returns the current date and time.

```
Ex:  select current_timestamp();
| current_timestamp() |
+---------------------+
| 2021-04-20 10:11:39 |
```

♦ **Date_add or Adddate:** Display a date value after adding "n" number of year/month/days to the specified date.

♦ **Date_sub or Subdate:** Display a date value after subtracting "n" number of year/month/days to the specified date.

```
Syntax:  date_add (date, interval expr unit)   or   adddate (date, interval expr unit)
         date_sub  (date, interval expr unit)   or  (date, interval expr unit)
Ex:  select date_add('2021-04-30', interval 1 month) as required_date;
     select date_add('2021-04-30', interval 1 year) as required_date;
select date_add('2021-04-30', interval 10 day) required;  or  select date_add('2021-04-30', interval 10 day);
     +---------------+
     | required_date |
     +---------------+
     | 2021-05-10    |
select date_sub('2021-04-30', interval 1 month); or  select subdate('2021-04-30', interval 1 month);
     +-----------------------------------------+
     | date_sub('2021-04-30', INTERVAL 1 month) |
     +-----------------------------------------+
     | 2021-03-30                              |
```

♦ **datediff():** returns the number of days between two dates. This function only calculates the date portion from each expression.

```
Syntax:  datediff (date1 , date2)
Ex:  select datediff('2021-04-30','2021-01-01');    or  select datediff(current_date,'2021-01-01');
+-------------------------------------+
| datediff('2021-04-30','2021-01-01') |
```

♦ **extract():** EXTRACTs a year , month or day of a given date

```
Syntax:  extract (year | month | day | hour | minute | second  from  date)
Ex: select extract (year from current_date);
     +--------------------------------+
```

```
| extract (year from current_date) |
+---------------------------------+
|                    2021         |
```

♦ **last_day:** returns the last day of the corresponding month for a date. If the date is invalid, the function returns NULL.

```
Syntax: last_day(date)
Ex:  select last_day('2021-04-01');
+-----------------------+
| last_day('2021-04-01') |
+-----------------------+
| 2021-04-30            |
```

♦ **makedate:** returns a date by taking a value of a year and a number of days. The number of days must be greater than 0 otherwise a NULL will be returned.

```
Syntax: makedaye(year, days of year)
Ex:  select makedate(2021,180);    --180 days
+--------------------+
| makedate(2021,180) |
+--------------------+
| 2021-06-29         |
```

➢ **Control Flow Functions and Expression**

The control flow function evaluates the condition specified in it. The output generated by them can be a true, false, static value or column expression. We can use the control flow functions in the SELECT, WHERE, ORDER BY, and GROUP BY clause. Following are the most common functions:

1. The IF functions
2. The CASE expression
3. The IFNULL function
4. The NULLIF function

Student table

ID	Stud_Name	Class_Room	Grade
1	Niraj	A	A+
2	Nayantara	A	A+
3	Bharti	A	B+
4	Diksha	B	A+
5	Hemant	B	D-
6	Usha	A	A+
7	Shailja	B	D
8	Deep	A	B+
9	Omkar	C	A+
10	Rajesh	NULL	NULL
11	Vinesh	NULL	NULL

Marks table

ID	Stud_id	Subjct	Marks
1	3	OS	100
2	1	Linux	80
3	1	DBMS	90
4	2	Linux	45
5	2	OS	50
6	2	DBMS	35
7	3	DBMS	35

♦ **IF() function**

The IF function returns the value based on the condition specified within the function.

```
Syntax:  SELECT IF (condition, if_condition_true, if_condition_false)
```

The **IF** function evaluates the values returned by the SELECT statement with the specified **condition**. If the function evaluates as TRUE, then it returns the value specified in ***if_condition_true***. If the function evaluates as **FALSE,** then it returns the value specified in the ***if_condition_false***.

```
Ex:  select if('nisarg'='nirali','True', 'False') as 'Simple IF statement';
```

```
+---------------------+
| Simple IF statement |
+---------------------+
| False               |
+---------------------+
select if('nisarg'='nisarg', 'True', 'False') as 'Simple IF statement';
+---------------------+
| Simple IF statement |
+---------------------+
| True                |
+---------------------+
```

- **Use IF to Print specific text in the output**

Two rows in the Student table with NULL values in grade and class columns. we want to show the ***Not available*** in the output instead of NULL value.

```
Ex:  Select stud_name grade, classroom from student;
     Select stud_name, IF (Class_room IS NULL, 'Not Applicable', Class_room) as 'Class_room',
            IF(grade IS NULL, 'Not Applicable', grade)as 'Grade' from student;
Ex: Print Brilliant Student for the student who has an A+ grade, else prints Average Student.
    select stud_name , IF(Grade='A+', 'Brilliant Student', 'Average Student') as 'Student Grade'
   from student;
Ex: total number of students with A+, B+, and D- grades using count function
    select
count(if(Grade='A+',1,NULL)) 'Brilliant Student',
count(if(Grade = 'B+', 1, NULL)) 'Average Student',
count(if(Grade = 'D-', 1, NULL)) 'Weak Student' from student;
```

♦ **CASE expression**

The case expression is used to implement the IF ELSE logic in the query. The CASE expression can be used in the SELECT, WHERE, and ORDER BY clause. There are two type of CASE expression.

1. The simple CASE expression
2. The searched CASE expression

1. **Simple CASE expression:** The simple CASE expression matches the input with the values for equality and returns the corresponding result.

```
Syntax:  SELECT
         Case  <expression>
         when <condition 1>  then output_1
         when <condition 2>  then output_2
         ..
         Else  <else_output>
         End
Ex: Display list of the students, their grades, and grade description.
    select stud_name, grade,
    case grade
    when 'A+' then 'Intelligent'
    when 'B+' then 'Average Student'
    when 'D' then 'Weak student'
    else 'No Grade' end as 'grade description' from student;
```

2. Searched CASE expression

The searched case expression evaluates the expression specified in the WHEN clause and returns the corresponding value.

Ex: to fill the name of the parents based on the name of the surname of the student based on following condition

1. If the student's surname is **Chauhan**, then the father's name must be **Hasmukh Chauhan**
2. If the student's surname is **Bhatt**, then the father's name must be **Jivan Bhatt**
3. If the student's surname is **Upadhyay,** then the father's name must be **Lalshankar Upadhyay**

```
select stud_name,
case
        when stud_name like '%Chauhan%' Then 'Hasmukh Chauhan'
        when stud_name like '%Bhatt%' then 'Jivan Bhatt'
        when stud_name like '%Upadhyay%' then 'Lalshankar Upadhyay' end as ParentName,
        Class_room,Grade from student where Class_room is not null;
```

- **IFNULL function**

This function accepts two parameters as an expression. If expression one is not NULL then it will return the first expression otherwise it will return the second expression. Arguments can be literal values or expressions.

Syntax: select IFNULL (NULL, expr2);

Ex: finds the NULL value in grade column and print 'not available' else we value of the column.

```
select stud_name, IFNULL(Grade, 'Not available') as Grade from student;
```

- **NULLIF function**

This function takes two parameters as an expression. It will return NULL if expr1=expr2 will return TRUE otherwise it will return expr1.

Syntax: select NULL IF (expr1, expr2);

Ex:
```
select NULLIF('Nisarg','Nirali');
select NULLIF('Nisarg', 'Nisarg');
```

MySQL Clause:

where Clause

- The WHERE clause is used to filter records.
- The WHERE clause is used to extract only those records that fulfill a specified condition.

Syntax: select select_list from table_name where search_condition;

1. WHERE clause with 'equal' operator

To find employees whose job titles are Salesman

```
select * from emp where job='salesman';
```

2. WHERE clause with 'and' operator

To find employees whose job titles are Salesman and dept_no are 50:

```
select * from emp where job='salesman' and dno=50;
```

3. WHERE clause with 'or' operator

To find employees whose job titles are Salesman or employees who locate dept_no are 50:

```
select * from emp where job='salesman' or dno=50;
```

4. WHERE clause with 'between' operator

Finds employees who locate in department whose dept_no is from 10 to 30:

```
select * from emp where dept_no between 10 and 30;
```

5. WHERE clause with 'like' and 'in' operator

Finds employees whose last names end with the string 'AN':

```
select * from emp where ename like '%AN';
```

Find employees who locate in the departmrnt with dept_no 10.

```
select * from emp where dno in (10,20,30);
```

comparison operators that you can use to form the expression in the WHERE clause.

➢ Distinct Clause

- DISTINCT clause is used to remove duplicate records from the table and fetch only the unique records.
- The DISTINCT clause is only used with the SELECT statement.

```
Syntax:  select  distinct <column_list> from  table_name;
```

Distinct clause to select unique last names from the employees table

```
select distinct ename from  emp;
```

- You can use the DISTINCT clause with an aggregate function e.g., SUM, AVG, and COUNT, to remove duplicate rows before the aggregate functions are applied to the result set.

```
select count (distinct city) from emp where state='Maharashtra';
```

➢ From Clause

The FROM Clause is used to select some records from a table. It can also be used to retrieve records from multiple tables using JOIN condition.

```
Syntax:   select *  from  table1  ;
For Join operation:  FROM table1   [ { INNER JOIN  | LEFT [OUTER] JOIN| RIGHT [OUTER] JOIN } table2
                          ON table1.column1 = table2.column1 ]
Example:  Retrieve data from one table
                    select * from emp;
              Retrieve data from two tables with inner join
                 SELECT dept.dname, emp.ename FROM emp INNER JOIN dept ON dept.dno = emp.dno;
```

➢ Order By Clause

The order by clause is used for sorting data in ascending and descending order based on one or more columns. Some databases sort query results in ascending order by default.

```
Syntax: select <column    _list> from  <table_name> where  <conditions> order by  [asc | desc];
            Sort the result in ascending order by name and salary.
                select * from emp order by ename, salary;
          To sort the result  in descending order by name.
                select * from emp order by name desc;
```

- The ORDER BY clause allows you to sort data using a custom list by using the FIELD() function.

Suppose that you want to sort the sales orders based on their statuses in the following order:

- In Process
- On Hold
- Canceled
- Resolved
- Disputed
- Shipped

```
select orderNumber, status from orders  order by  field(status, 'In Process','OnHold', 'Cancelled','Resolved',
'Disputed','Shipped');
```

➢ Group By Clause

- Group by Statement is used to organize similar data into groups with the help of some functions. i.e. if a particular column has same values in different rows then it will arrange these rows in a group.
- GROUP BY returns only one result per group of data
- A query containing a group by clause is processed in the following way:
  1. Select all rows that satisfy the condition specified in the where clause.
  2. Discard all groups that do not satisfy the condition in the having clause.
  3. Apply aggregate functions to each group.

```
Syntax:  select column1, aggregate function_name (column2) from table_name where condition group
         by column1, column2;
```

1. **Aggregate function_name**: Name of the function. Example- SUM(),AVG().
2. **table_name**: Name of the table.

3. **condition**: Condition used.

Example: select dno, sum(sal) from emp group by dno;

Table Store_Info

create table store_info(Store_Name char(10),Product_ID int,Sales int,Txn_Date varchar(15));

insert into store_info values

('Sangola',1,1500,'Jan-05-1999'),('Sangola',2,500,'Jan-05-1999'),('Pune',1,250,'Jan-07-1999'),

('Sangola',1,300,'Jan-08-1999'),('Sangli',1,700,'Jan-08-1999');

Store_Name	Product_ID	Sales	Txn_Date
Sangola	1	1500	Jan-05-1999
Sangola	2	500	Jan-05-1999
Pune	1	250	Jan-07-1999
Sangola	1	300	Jan-08-1999
Sangli	1	700	Jan-08-1999

⦿ **Group By a single column :** To find total sales for each store:

Select store_name, sum(sales) from store_info group by store_name;

Store_Name	SUM(Sales)
Sangola	2300
Pune	250
Sangli	700

⦿ **Group By multiple columns:** To have two or more columns associated with GROUP BY.

To find total sales for each product at each store:

Select store_name, product_id, sum(sales)from store_info group by store_name,product_id;

Store_Name	Product_ID	SUM(Sales)
Sangola	1	1800
Sangola	2	500
Pune	1	250
Sangli	1	700

⦿ **GROUP BY multiple columns and multiple functions**

To find total sales and the average sales for each product at each store:

Select store_name, product_id, sum(sales), avg(sales) from store_info group by store_name, product_id;

Store_Name	Product_ID	SUM(Sales)	AVG(Sales)
Sangola	1	1800	900
Sangola	2	500	500
Pune	1	250	250
Sangli	1	700	700

⦿ **Group by month / date / week:** To find total daily sales from Store_Info.

select txn_date, sum(sales)from store_info group by txn_date order by txn_date;

Txn_Date	SUM(Sales)
Jan-05-1999	2000
Jan-07-1999	250
Jan-08-1999	1000

Total sales for both Sangola and Sangli stores.

Select store_name, sum(sales) from store_info where store_name in('Sangola', 'Sangli') group by store_name;

Store_Name	SUM(Sales)
Sangola	2300
Sangli	700

➢ HAVING Clause

- We know that WHERE clause is used to place conditions on columns but we want to place conditions on groups then, use HAVING clause.
- The **HAVING** clause is used to filter group of rows based on a specified condition. It specifies the search condition for the group or aggregate. HAVING clause is attached with **GROUP BY** clause. The HAVING clause works like the WHERE clause.
- The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

Syntax: **select** column1, function_name(column2) **from** table_name **where** condition **group by** column1, column2 **having** aggregate_function (<expression>) <operator><value>**order by** column1, column2;

1. **function_name**: Name of the function used for example, SUM() , AVG().
2. **table_name**: Name of the table.
3. **Condition**: Condition used.

Employee

EmployeeID	Ename	DeptID	Salary
1001	John	2	4000
1002	Anna	1	3500
1003	James	1	2500
1004	David	2	5000
1005	Mark	2	3000
1006	Steve	3	4500
1007	Alice	3	3500

SELECT *DeptID, AVG(Salary)*
FROM *Employee*
GROUP BY *DeptID;*

GROUP BY Employee Table using DeptID

DeptID	AVG(Salary)
1	3000.00
2	4000.00
3	4250.00

SELECT *DeptID, AVG(Salary)*
FROM *Employee*
GROUP BY *DeptID*
HAVING *AVG(Salary) > 3000;*

HAVING

DeptID	AVG(Salary)
2	4000.00
3	4250.00

Employee table is grouped based on DeptID column and these grouped rows filtered using HAVING Clause with condition AVG(Salary) > 3000.

Example: select dno, count(eno) from emp group by dno having count(eno)>2;

To see only the stores with sales over Rs.1500.

select store_name, sum(sales) from store_info group by store_name having sum(sales)>1500;

Store_Name	SUM(Sales)
Sangola	2300

Total sales for both Sangola and Sangli stores.

Select store_name, sum(sales) from store_info group by store_name having store_name in('Sangola', 'Sangli');

Store_Name	SUM(Sales)
Sangola	2300
Sangli	700

WHERE	HAVING
Implemented in row operations.	Implemented in column operations.
Applied to Single row	Applied to Summarized row or groups.
It only fetches the particular data from particular rows according to the condition.	At first, complete data is fetched then separated according to the condition.
Aggregate Functions cannot appear in WHERE clause.	Aggregate Functions can appear in HAVING clause.
Used with SELECT and other statements such as UPDATE, DELETE etc.	Used with SELECT statement. Can't be used other statements.
Act as a Pre-filter	Act as a Post-filter
GROUP BY Comes after WHERE.	GROUP BY Comes before HAVING.

- **Rollup Clause**

If we want to understand the ROLLUP modifier, we must have to know what is grouping set. A grouping set is a set of columns that we want to group to get the resultant output.

To find total sales and the average sales for each product at each store:

```
Select store_name, product_id, sum(sales), avg(sales) from store_info group by store_name, product_id
with rollup;
```

MySQL constraints

♣ Applying data constraints- column level and table level
♣ Types of Data constraints- I/O constraints, Business rule constraints
- NOT NULL constraint, UNIQUE constraint, CHECK constraint
- Primary key and Foreign key constraint
- Disable foreign key checks

♣ Adding, Modify and drop constraints using alter table command

---

MySQL constraint is used to specify the rule that allows or restricts what values/data will be stored in the table. They provide a suitable method to ensure data accuracy and integrity inside the table. It also helps to limit the type of data that will be inserted inside the table. If any interruption occurs between the constraint and data action, the action is failed.

- Constraints are the rules. These can be specified at the time of table creation or can be added after table creations by using alter table statement.
- Constraints prevent the table from deletion, if there is any dependency

Applying data constraints

Constraints can be divided into following two types:

1) The constraints can be specified immediately after the column definition is called column-level definition.

2) The constraints can be specified after all columns and end of table is called table-level definition.

Data Integrity:

- **Entity Integrity** ensures that no duplicate rows in a table. Ex: Unique, Primary Key
- **Domain Integrity** enforces valid entries for a given column by restricting the type, the format, or the range of possible values. Ex: check, Null, Not Null
- **Referential integrity** ensures that rows cannot be deleted, which are used by other records. Ex: Foreign Key
- **User-Defined Integrity** enforces some specific business rules on entity, domain, or referential integrity customer.

Constraints are:

- **Not Null Constraint:** Ensures that a column have not NULL value.
- **Default Constraint:** Provides a default value for a column when none is specified.
- **Unique Constraint:** Ensures that all values in a column are different (unique).
- **Primary Key:** Uniquely identified each rows/records in a database table.
- **Foreign Key:** Uniquely identified a rows/records in any another database table.
- **Check Constraint:** Ensures that all values in a column must satisfy certain conditions.

Not Null Constraint

- By default, table columns are null, You should manually define NOT NULL constraint
- NOT NULL constraint enforces a column to not accept NULL values. i.e. column cannot have NULL or empty values.
- NOT NULL constraint prevents inserting NULL values into a column.
- If you try to insert or update a NULL value in the column, then database engine will reject the change and issue an error.
- NOT NULL constraint applied only at column level not table level.
- When a column is defined as not null, that column becomes a mandatory column. It force to user enter data into the column.

Principles of NULL values

- NULL value is different from a blank or zero.
- NULL means unknown or missing information.
- NULL value can be inserted into the columns of any Data type.

Syntax: column_name data_type NOT NULL;

Example: To create an emp_info table and enforces "Name" column to not null

```
create table emp_info (emp_no int(3) primary key, emp_name char(10) not null, address varchar(20));
desc emp_info;
| Field        | Type        | Null  | Key | Default | Extra |
+----------+----------+------+-----+---------+-------+
| emp_no     | int(3)        | NO   | PRI | NULL    |        |
| emp_name | char(10)     | NO   |       | NULL    |        |
| address     |varchar(10)  | YES  |       | NULL    |        |
+----------+----------+------+-----+---------+-------+
```

⦿ **Add NOT NULL constraint in existing table column**

```
Alter table emp_info modify address varchar(30) not null;
Desc emp_info;
| Field        | Type       | Null | Key | Default | Extra |
+----------+----------+------+-----+---------+-------+
| emp_no     | int(3)      | NO   | PRI | NULL    |        |
| emp_name | char(10)  | NO   |       | NULL    |        |
| address    varchar(10)  | NO   |       | NULL    |        |
+----------+----------+------+-----+---------+-------+
```

⦿ **Drop NOT NULL constraint from existing table column**

```
Alter table emp_info modify address varchar (30) null;
Desc emp_info;
```

Unique Key Constraint

- A UNIQUE constraint is an integrity constraint that ensures values in a column or group of columns to be unique.
- The UNIQUE constraint is used for uniquely identify each record in a database. It requires that each value in a column should be unique.
- It provides uniqueness guarantee of a column or set of columns. It cannot allow duplicate values but **unique key allow NULL (blank) values.**
- You can assign many unique key constraints in one table.
- UNIQUE constraint can be applied at column level and table level.

Syntax: Column-Level

```
create table table_name (
  column_name data_type,
  column_name data_type UNIQUE,
  ... );
```

Table-Level

```
create table table_name (
  column_name1  data_type (size),
  column_name2  data_type (size),
  ...,
  UNIQUE (column_name1, column_name2) );
```

Example: To create an emp_info table and enforces "Name" column to not null

unique key at column level	Unique key at table level
create table emp_info (emp_no int(3) primary key, **email_id varchar(20) unique,** mobile_no varchar(15));	create table emp_info (emp_no int(3) primary key, email_id varchar(20), mobile_no varchar(15), **unique(email_id, mobile_no));**

```
desc emp_info;
+----------+----------+------+-----+---------+-------+
| Field        | Type        | Null  | Key | Default | Extra |
+----------+----------+------+-----+---------+-------+
| emp_no     | int(3)        | NO   | PRI | NULL    |        |
| email_id   | varchar(20) | YES  | UNI | NULL    |        |    --Unique key Assigned
| mobile_no| varchar(15) | YES  |       | NULL    |        |
```

Now we are inserting record into this table to check unique key how to work

Example: try to insert same email id fire error for unique constraint violated.

```
insert into emp_info values(1,'omsai@gmail.com',9874563210);
insert into emp_info values(2,'omsai@gmail.com',9870263210);
ERROR 1062 (23000): Duplicate entry 'omsai@gmail.com' for key 2
```

⦿ **UNIQUE KEY constraint on ALTER TABLE:**

- Defining a unique key constraint on single column:

```
alter table student add unique (PRN_no);
```

- Defining a unique key constraint on multiple columns:

```
alter table student add constraint uniqcon_student unique (email_id, mobile_no);
```

Primary Key Constraints:

MySQL primary key is a single or combination of the field, which is used to identify each record in a table uniquely. MySQL automatically creates an index named "**Primary**" after defining a primary key into the table. Since it has an associated index, we can say that the primary key makes the query performance fast.

- Primary Key apply on columns for uniquely identifies each record from the table.
- Primary Key constraint column cannot accept duplicate data and NULL values.
- Each table can have only ONE primary key.
- MySQL does not allow us to insert a new row with the existing primary key.
- A primary key is assigned one or more columns in a table is called **composite primary key.**
- It defines a mandatory column and data through the column must be unique.
- Primary key constraint create automatic index file for identifying unique records.
- Primary key constraint can be defined at the column and table level.
- Primary key constraint combination of NOT NULL and UNIQUE Constraints

When we specify a primary key constraint for column, database engine automatically creates a unique index file for identify unique records.

at table level (primary key for single columns)	At (single) column level
CREATE TABLE table_name (Column1 Column _definition, Column2 Column_definition, ... PRIMARY KEY (column_name(s)));	CREATE TABLE table_name(Column1 data type(size) PRIMARY KEY, Column2 data type (size), ...);
create table doctor (did int(5), dname char(20), age int(2), contact_no varchar(15), primary key(sid));	create table doctor (did int (5) primary key, dname char(20), age int(2), contact_no varchar(15));

⦿ **Primary key with Auto_increment attribute:**

When you insert a new row into the table, the primary key column can also use the **AUTO_INCREMENT** attribute to generate a sequential number for that row automatically.

```
create table patient( pid int (5) auto_increment primary key,
pname char(20));

insert into stud (pname) values('Abhi');
insert into stud (pname) values('Nayan');
select * from patient;
```

⦿ **Primary key on single column:**

```
alter table users_info add primary key (uid);
```

⦿ **Primary key on multiple columns:**

```
alter table users_info add constraint pk_users_id primary key (uid, username);
```

⦿ **Drop Primary Key constraint**

```
alter table stud drop primary key;
alter table stud drop constraint pk_users_id ;
```

⦿ **Enable or Disable a Primary Key**

```
alter table students disable constraint pk_users_id;
alter table students enable constraint pk_users_id;
```

Foreign Key (referential) Constraint

- A foreign key is a column or group of columns in a table that links to a column or group of columns in another table.
- Foreign key represents relationship between one or more tables. FOREIGN KEY applies on columns that references values in another table column.
- FOREIGN KEY constraints also known as relationship (referential) constraints.
- A foreign key is a column whose values are derived from the primary key of the other table.
- FOREIGN KEY constraint applied column must have same data type as they reference on another table column.

Syntax: FOREIGN KEY (column_name, ...) REFERENCES parent_table (colunm_name,...)

OR

[CONSTRAINT constraint_name] FOREIGN KEY [foreign_key_name] (column_name, ...) REFERENCES parent_table (colunm_name,...) [ON DELETE reference_option] [ON UPDATE reference_option]

⦿ **MySQL has five reference options:**

MySQL fully supports three actions: RESTRICT, CASCADE and SET NULL.

- **CASCADE:** if a row from the parent table is deleted or updated, the values of the matching rows in the child table automatically deleted or updated.
- **SET NULL**: if a row from the parent table is deleted or updated, the values of the foreign key columns in the child table are set to NULL.
- **RESTRICT:** if a row from the parent table has a matching row in the child table, MySQL rejects deleting or updating rows in the parent table. If you don't specify the ON DELETE and ON UPDATE clause, the default action is RESTRICT.
- **NO ACTION:** is the same as RESTRICT.
- **SET DEFAULT:** is recognized by the MySQL parser.

Create two tables Customer and bill:

```
create table customer (cust_id int auto_increment primary key,
        cust_name char(10));

 create table bill (bill_no int auto_increment primary key,
        bill_amount int,
        cust_id int,
 foreign key (cust_id) references cust(cust_id));
mysql> desc customer;
+-----------+---------+------+-----+---------+---------------+
| Field     | Type    | Null |Key  | Default | Extra |
+-----------+---------+------+-----+---------+---------------+
| cust_id   | int(11) | NO   | PRI | NULL    |       |
| cust_name | char(10)| YES  |     | NULL    |       |

mysql> desc bill;
+-----------+---------+------+-----+---------+-------+
| Field     | Type    | Null | Key | Default | Extra |
+-----------+---------+------+-----+---------+-------+
| bill_no   | int(11) | NO   | PRI | NULL    |       |
| bill_amount | int(11) | NO  |     | NULL    |       |
| cust_id   | int(11) | YES  | MUL | NULL    |       |      --- Foreign key column
```

The cust_id in the bill table is the foreign key column that refers to the cust_id column in the customer table.

- **Restrict Action.**

1) Insert two rows into the customer table:

```
insert into customer (cust_name) values ('smita'), ('sanjay');
select * from customer;
```

2) Insert a new row into the bill table:

```
insert into bill (bill_amount, cust_id) values(2000,1);
```

It works because the cust_id 1 exists in the customer table.

3) Insert a new row into the bill table with a cust_id value does not exist in the customer table:

```
insert into bill (bill_amount, cust_id) values(4000,3);
```

MySQL issued the following error:

```
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`bcs`.`bill`,  FOREIGN KEY (`cust_id`) REFERENCES `customer` (`cust_id`) ON DELETE RESTRICT ON UPDATE RESTRICT)
```

4) Update the value in the cust_id column in the customer table to 100:

```
update customer set cust_id = 100 where cust_id = 1;
```

MySQL issued this error:

```
Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`bcs`.`bill`, CONSTRAINT FOREIGN KEY (`cust_id`) REFERENCES `customer` (`cust_id`) ON DELETE RESTRICT ON UPDATE RESTRICT)
```

Because of the RESTRICT option, you cannot delete or update cust_id 1 since it is referenced by the bill_no 1 in the bill table.

- **Cascade Action:** show how **on update cascade** and **on delete cascade** actions work.

1) Drop the bill table:

```
drop table bill;
```

2) Create the bill table with the on update cascade and on delete cascade options for the foreign key:

```
create table bill(
   bill_no int auto_increment primary key,
   bill_amount int(11),
   cust_id int,
   foreign key (cust_id) references customer(cust_id) on update cascade on delete cascade  ) ;
```

3) Insert four rows into the bill table:

```
insert into bill(bill_amount, cust_id) values     (2000, 1),
                                                   (2500,1),
                                                  (3000,2),
                                                 (4000,2);
    select * from bill;
```

4) Update cust_id 1 to 100 in the customer table:

```
update customer set cust_id = 100 where cust_id = 1;
select * from customer;
select * from bill;
```

you can see, two rows with value 1 in the cust_id column of the bill table were automatically updated to 100 because of the ON UPDATE CASCADE action.

5) Delete cust_id 2 from the customer table:

```
delete from customer  where cust_id = 2;
select * from customer;
select * from bill;
```

All bill with cust_id 2 from the bill table were automatically deleted because of the on delete cascade action.

➢ Set Null Action

How the on update set null and on delete set null actions work.

1) Drop both customer and bill tables:

```
drop table if exists customer;
drop table if exists bill;
```

2) Create the customer and bill tables:

```
create table customer(
   cust_id int auto_increment primary key,
   cust_name char(10));

create table bill(
   bill_no int auto_increment primary key,
   bill_amount int,
   cust_id int,
   foreign key (cust_id) references customer(cust_id) on update set null  on delete set null );
```

The foreign key in the bill table changed to on update set null and on delete set null options.

3) Insert rows into the customer table:

```
insert into customer(cust_name) values ('smita'),  ('sanjay');
```

4) Insert rows into the bill table:

```
insert into bill(bill_amount, cust_id) values    (2000, 1),
                                                 (2500,1),
                                                 (3000,2),
                                                 (4000,2);
```

5) Update cust_id from 1 to 100 in the customer table:

```
update customer set cust_id = 100 where cust_id = 1;
select * from customer;
select * from bill;
```

The rows with the cust_id 1 in the bill table were automatically set to null due to the on update set null action.

6) Delete the cust_id 2 from the customer table:

```
delete from customer where cust_id = 2;
select * from bill;
```

The values in the cust_id column of the rows with cust_id 2 in the bill table were automatically set to null due to the on delete set null action.

⊙ To add foreign key in existing table

When you adding foreign key constraint, SQL check any existing data violate the foreign key constraint or not. If not violate constraint added successfully otherwise you have to update invalid data to prevent foreign key constraint violating.

```
alter table bill add foreign key (cust_id) references customer(cust_id);
```

⊙ Drop Foreign Key Constraints

To drop a foreign key constraint, you use the alter table statement:

```
alter table table_name  drop foreign key constraint_name;
```

To obtain the generated constraint name of a table, you use the show create table statement:

```
show create table table_name;
```

Example, to see the foreign keys of the bill table, you use the following statement:

```
show create table bill;
```

Drops the foreign key constraint of the bill table:

```
alter table bill  drop foreign key;
```

To ensure that the foreign key constraint has been dropped, you can view the structure of the bill table:

```
show create table bill;
```

⦿ Disabling and Enabling Foreign Key Checks

it is very useful to disable foreign key checks e.g., when you import data from a csv file into a table. if you don't disable foreign key checks, you have to load data into a proper order i.e., you have to load data into parent tables first and then child tables, which can be tedious.

Disable foreign key

```
set foreign_key_checks = 0;
```

Enable foreign key

```
set foreign_key_checks = 1;
```

Primary key	Foreign key
Primary key cannot be null	Foreign key can be null.
Primary key is always unique. It uniquely identify a record in a table	Foreign key can be duplicated Because foreign key is a field refer primary key in another table.
only one primary key in the table	More than one foreign key in the table.

Check Constraints:

- Check constraint is used to check the value which is entered into a record. It is used to define condition which each row must satisfy.
- If the condition value evaluates to false, then the record violates the constraint and you cannot enter it into the table.
- We cannot define check constraint in SQL view, sub queries or sequences.
- It performs check on the values, before storing them into the database. It's like condition checking before saving data into a column.
- The constraint can be applied for a single column or a group of columns.

Syntax: [CONSTRAINT constraint name] CHECK (condition)

⦿ Define CHECK constraint at column level with other column attributes

```
create table student_info(stu_code varchar(6) primary key check (stu_code like 'st%'),
                name varchar (30) check(name=upper(name)),
                city varchar (30) check(city in('mumbai','pune','delhi','chennai')),
               scholarship int(5) check (scholarship between 5000 and 20000),
               class varchar(15));

insert into student_info values ('st001','NAYANTARA','pune',8900,'ecs-II');
```

We are creating new **student_info** table name with following check constraints:

1. Values inserted into **stu_code** column must be start with the lower letter **'ST'**.
2. Values inserted into **name** column must be capitalized.
3. Values inserted into **city** column only allow 'Mumbai','Pune','Delhi','Chennai' as valid legal values.
4. Values inserted into scholarship column between 5000 and 20000.

⦿ CHECK constraint apply in table level.

```
Create table student_info(stu_code varchar(10) primary key,
                Name varchar(30),
                city varchar (30), scholarship int(5), class varchar(10),
                check(stu_code like'st%'),
                check (name = upper(name)),
                check(city in('mumbai','pune','delhi','chennai')),
                check (scholarship between5000and20000));

insert into student_info values ('st002','NAYAN','delhi',9000,'bca-II');
```

⦿ To add CHECK constraint in existing table column

```
alter table student_info add constraint check_namecheck(class=upper(class));
```

⦿ To drop CHECK constraint in existing table column

```
Alter table student_info drop constraint check_name;
```

Default Constraint:

- DEFAULT constraint is used to set a default value for a column. When the insert into statement does not provide a specific value.
- Default constraint helps you assign a particular default value to every row of a column.
- DEFAULT constraint specified only at column level.

```
Syntax: create table table_name (
            column1 column1_definition DEFAULT default_value,
            column2 column2_definition,
            ...
            );
Example: Create table stud_info (roll_no int(3) primary key,
                stud_name char(30) not null,
                class varchar(15) default 'M.Sc.(CS)',
                fee  int(5));
      insert into stud (roll_no,stud_name,fee) values (1,'Omkar',5000);
      select * from stud;
      +---------+-----------+-----------+----------+
      | roll_no | stud_name | class     | fee |
      +---------+-----------+-----------+----------+
      |     1   | Omkar     | M.Sc.(CS) | 5000 |
```

Alter table add DEFAULT constraint with default value in existing table column

```
Syntax:  alter table table_name alter column_name set default default_value;
Example:  Alter table stud_info modify fee int(5) default 15000;
            desc stud_info;
      +-----------+-------------+------+-----+-----------+-------+
      | Field     | Type        | Null | Key | Default   | Extra |
      +-----------+-------------+------+-----+-----------+-------+
      | roll_no   | int(3)      | NO   | PRI | NULL      |       |
      | stud_name | char(30)    | NO   |     | NULL      |       |
      | class     | varchar(15) |YES   |     | M.Sc.(CS) |       |  --Default value columns
      | fee       | int(5)      | YES  |     | 15000     |       |
```

To remove DEFAULT constraint in existing table column

If you need to remove default value for column.

```
Syntax:  ALTER TABLE table_name ALTER column_name DROP DEFAULT;
Example:  Alter table stud_info alter fee drop default;
            desc stud_info;
      +-----------+-------------+------+-----+-----------+-------+
      | Field     | Type        | Null | Key | Default   | Extra |
      +-----------+-------------+------+-----+-----------+-------+
      | roll_no   | int(3)      | NO   | PRI | NULL      |       |
      | stud_name | char(30)    | NO   |     | NULL      |       |
      | class     | varchar(15) |YES   |     | M.Sc.(CS) |       |
      | fee       | int(5)      | YES  |     | NULL      |       |  --Default value of columns removed.
```